

Cache & Redis

150 Interview Q&A — SDE2 Edition

SDE2 Interview Preparation Series

iPad Edition · Dark Code Blocks · Syntax Highlighting

Caching SDE2 Interview Guide — 100 Questions & Answers

> **Focus:** Cache Fundamentals, Redis Internals, Eviction, Patterns, Distributed Caching, Go Integration | **Level:** SDE2

Table of Contents

1. [Cache Fundamentals](#1-cache-fundamentals) — Q1–Q20
 2. [Redis Data Structures & Internals](#2-redis-data-structures--internals) — Q21–Q40
 3. [Eviction Policies & TTL](#3-eviction-policies--ttl) — Q41–Q50
 4. [Cache Patterns & Strategies](#4-cache-patterns--strategies) — Q51–Q65
 5. [Distributed Caching & Redis Cluster](#5-distributed-caching--redis-cluster) — Q66–Q80
 6. [Advanced Topics & Go Integration](#6-advanced-topics--go-integration) — Q81–Q100
-

1. Cache Fundamentals

Q1

What is a cache and why do we use it?

Difficulty: Easy

A cache stores copies of data in fast-access storage to reduce latency and load on slower backing stores (databases, APIs, disk).

```
Without cache:      With cache:
User → App → DB (50ms) → User  User → App → Cache (0.5ms) → User
                                ↓ miss
                                App → DB (50ms) → Cache → User
```

Benefits: Reduce DB load, lower latency (sub-millisecond vs tens of ms), absorb traffic spikes, reduce cost (fewer DB ops).

When NOT to cache: Financial balances (stale = wrong), user-specific secrets, rapidly changing data with strict consistency requirements.

Q2

What are cache hit, miss, and hit ratio?

Difficulty: Easy

```
Cache hit: request served from cache (fast, cheap)
Cache miss: cache doesn't have it → fetch from DB → populate cache
Hit ratio: hits / (hits + misses) × 100%

Target hit ratio: >90% for most production caches
If hit ratio < 80%: cache is not helping much, reconsider strategy

Monitoring in Redis:
redis-cli INFO stats | grep keyspace
# keyspace_hits: 10000000
# keyspace_misses: 100000
# hit_ratio = 10M / (10M + 100K) ≈ 99%

Cold start: after deploy/restart, cache is empty → all misses
→ pre-warm critical keys before switching traffic
→ or accept brief latency spike on startup
```

Q3

What is cache latency and how does it compare to DB?

Difficulty: Easy

Storage latency comparison:

CPU L1 cache:	0.5 ns	
CPU L2 cache:	7 ns	
RAM:	100 ns	
Redis (local):	0.1-0.5 ms	← in-process or same host
Redis (remote):	1-5 ms	← cross-AZ
SSD (local):	0.1 ms	
PostgreSQL:	5-50 ms	← indexed query
PostgreSQL:	50-500 ms	← complex query / cold cache
HDD:	5-10 ms	
S3:	20-100 ms	

Rule of thumb: Redis $\approx$ 10-100 $\times$ faster than DB for cached reads

At 1M QPS: 50ms DB per query = 50,000 DB-seconds/sec = 50,000 cores needed
 0.5ms Redis per query = 500 Redis-cores needed

Q4

What are the layers of caching in a web application?

Difficulty: Easy

Browser cache	→ HTML, JS, CSS, images (304 Not Modified, Cache-Control)
CDN edge cache	→ static assets, public API responses (global PoPs)
API Gateway cache	→ response caching per route
Application cache	→ in-process (sync.Map, groupcache, bigcache)
Distributed cache	→ Redis / Memcached (shared across instances)
DB query cache	→ PostgreSQL shared_buffers, buffer pool
OS page cache	→ file system reads
Hardware cache	→ CPU + SSD caches

In-process vs distributed:

In-process: ~0ms (no network), not shared, lost on restart
 Distributed: ~1ms (network), shared, survives restarts
 L1 (in-process) + L2 (Redis) = best of both worlds

Q5

What is the difference between cache-aside, read-through, and write-through?

Difficulty:
Medium

Cache-Aside (Lazy Loading):

Read: check cache → miss → read DB → store in cache → return
 Write: write to DB → delete/update cache
 Pros: cache only what's needed, resilient to cache failure
 Cons: first read always cold, possible stale data

Read-Through:

App always reads from cache
 Cache fetches from DB on miss automatically (cache handles it)
 Pros: transparent to app
 Cons: cold start, cache lib must know DB schema

Write-Through:

App writes to cache → cache synchronously writes to DB
 Pros: cache always up to date
 Cons: write latency (2 writes), cache stores data never read

Write-Behind (Write-Back):

Write to cache only → async flush to DB
 Pros: very fast writes, absorbs bursts
 Cons: data loss if cache crashes before flush

Most common production: Cache-Aside reads + Delete on writes

Q6

What is cache invalidation and why is it hard?

Difficulty:
Medium

"There are only two hard things in Computer Science:
 cache invalidation and naming things." – Phil Karlton

Invalidation strategies:

1. TTL (Time-To-Live): expire after N seconds
 Simple, always eventually consistent
 Downside: stale for up to TTL duration
2. Delete on write: when DB is updated, delete cache key
 Consistent (cache repopulated on next read)
 Downside: thundering herd on popular keys
3. Event-driven: publish cache invalidation event to Kafka/Redis Pub/Sub
 Other services subscribe and delete local cache
 Downside: async, brief inconsistency window
4. Versioned keys: key = "user:42:v7" where v7 = current version
 Old versions auto-expire via TTL
 Downside: old versions waste memory until TTL

Best practice: delete key on write (not update) + short TTL as safety net
 Never update cached values directly – race conditions

Q7

What is the thundering herd problem?

Difficulty: Hard

Thundering herd (cache stampede):

Popular key expires → 1000 concurrent requests → all miss
→ 1000 requests hit DB simultaneously → DB overload → slow → more misses

Solutions:

1. Mutex / single-flight (most common):

Only first requester fetches from DB; others wait for result

Go: golang.org/x/sync/singleflight

```
var group singleflight.Group
result, err, _ := group.Do(key, func() (interface{}, error) {
    return db.GetUser(id)
})
```

2. Jittered TTL:

TTL = base_ttl + random(0, jitter) e.g., 300s ± 30s

Prevents all keys expiring simultaneously

Simple, always use this

3. Background refresh (proactive):

Before key expires: background goroutine refreshes it

Client always gets cached value (no miss latency)

4. Stale-while-revalidate:

Serve stale data immediately

Trigger async background refresh

Client gets fast (possibly old) response

Production: singleflight + jittered TTL + background refresh for hottest keys

Q8

What is cache warming and why is it important?

Difficulty:
Medium

Cache warming: pre-populating cache before traffic hits
Prevents cold start performance degradation on deploys

When you need it:

- After Redis flush/restart
- New cluster node added
- After application deploy (if in-process cache cleared)
- After major schema change

Strategies:

1. Lazy warm (natural): first requests populate cache
Simple but first N requests are slow
OK for non-critical paths
2. Explicit pre-warm: load popular keys before serving traffic
Read top 10K products from DB → SET in Redis
Run before deploy completes
3. Replay warm: copy cache from production to staging
RDB snapshot → load into new Redis
4. Traffic replay: send shadow traffic before cutover

```
// Go: warm on startup
func warmCache(ctx context.Context, db *sql.DB, redis *redis.Client) error {
    products, _ := db.QueryContext(ctx, "SELECT id, data FROM products ORDER BY views DESC LIMIT 10000")
    pipe := redis.Pipeline()
    for products.Next() { pipe.Set(ctx, key, data, 10*time.Minute) }
    _, err := pipe.Exec(ctx)
    return err
}
```

Q9

What is a cache stampede vs a hotspot?

Difficulty:
Medium

Cache stampede: many simultaneous misses for ONE expired key
→ burst of DB requests for same key
Fix: singleflight, mutex, background refresh

Hotspot: one key accessed far more than others
→ single Redis instance handles all traffic for that key
→ single-threaded Redis gets overloaded for that key

Example: celebrity tweet being read by 1M followers/sec

Hotspot solutions:

Local in-process cache (L1): each app server caches hot key locally
→ 0ms read, no Redis load
→ short TTL (seconds) for freshness

Key replication: replicate hot key to multiple Redis slots
key:copy1, key:copy2, key:copy3 → read-only replicas
Route reads round-robin across copies

Redis read replicas: add Redis replica nodes for read-only scale
Primary: writes only
Replicas: reads only, replicate from primary

Q10

What is cache penetration?

Difficulty:
Medium

Cache penetration: querying for keys that don't exist in DB
→ every request misses cache (no value to cache)
→ every request hits DB
→ can be exploited as DoS attack (flood with random IDs)

Solutions:

1. Cache null values:

Key doesn't exist in DB → cache null with short TTL (30s)

SET user:99999 "null" EX 30

Next request: cache hit → null → return 404

Downside: wasted cache space

2. Bloom filter:

Probabilistic set membership: "is key 99999 in DB?"

No false negatives: if Bloom says NO → definitely not in DB → skip DB

False positives possible (0.1%): might check DB when not needed

Space-efficient: 10 bits per element at 1% false positive rate

Check Bloom filter → "definitely not" → return 404 (no DB hit)

Redis: RedisBloom module (BF.ADD, BF.EXISTS)

3. Rate limiting by IP/user:

Limit requests per key pattern to prevent DoS

Production: Bloom filter for large key spaces + cache nulls for small

Q11

What is a Bloom filter?

Difficulty: Hard

Bloom filter: space-efficient probabilistic set membership test
 "Is element X in the set?" → "Definitely NO" or "Probably YES"
 No false negatives, possible false positives

How it works:

Bit array of M bits, K hash functions
 Add element: set bits at positions $h_1(x)$, $h_2(x)$, ..., $h_k(x)$
 Query: check if ALL k positions are 1
 All 1 → probably in set (false positive possible)
 Any 0 → definitely NOT in set

Properties:

Cannot delete (use Counting Bloom Filter for deletions)
 False positive rate $\approx (1 - e^{(-kn/m)})^k$
 At 1% FPR: ~9.6 bits per element (1M elements = 1.2 MB)
 At 0.1% FPR: ~14.4 bits per element

Applications:

Cache penetration: skip DB for keys not in Bloom filter
 Cassandra: per-SSTable Bloom filter to avoid reading wrong file
 Chrome: malicious URL detection
 HBase, RocksDB: avoid disk reads for non-existent keys

```
// Go:
import "github.com/bits-and-blooms/bloom/v3"
filter := bloom.New(1000000, 5) // 1M items, 5 hash functions
filter.Add([]byte("user:42"))
exists := filter.Test([]byte("user:99999")) // false → skip DB
```

Q12

What is cache consistency and how do you handle it?

Difficulty: Hard

Cache consistency: cache reflects latest DB state

Levels of consistency:

Strong: cache always matches DB (too expensive)

Eventual: cache converges to DB state within TTL

Read-your-writes: user sees their own writes immediately

Practical approaches:

1. Write-through: update cache and DB atomically

Problems: write latency, race conditions

2. Delete-on-write: DB write → delete cache key

Next read: repopulate from DB

Safer than updating (avoids race conditions)

Race condition in cache update:

Thread1: read DB(v1) → Thread2: write DB(v2), delete cache

→ Thread1: write cache(v1) → STALE!

Race condition in cache delete:

Thread1: read DB(v1) → Thread2: write DB(v2), delete cache

→ Thread1: write cache(v1) → Thread3: reads cache(v1) STALE

But: solved by short TTL as safety net

3. TTL safety net: even if invalidation misses, TTL ensures eventual consistency

4. Cache-aside with version check:

Store version with cached data

Compare with DB version on read → invalidate if mismatch

Q13

What is the write-around pattern?

Difficulty: Easy

Write-Around: writes `go` directly to DB, bypassing cache
 Cache populated lazily on first read

When to use:

Data written once, rarely read (logs, audit events)
 Prevents cache pollution (caching data nobody will ever read)

Analogy: bulk `import` – don't cache 1M imported rows nobody will read

vs Write-Through:

Write-Through: every write → cache + DB (all writes cached)
 Write-Around: write → DB only (only reads populate cache)

Implementation:

```
// Write-Around:
func createAuditLog(log AuditLog) error {
    return db.Insert(log) // no cache write
}

// Read (Cache-Aside, lazy population):
func getAuditLog(id string) (*AuditLog, error) {
    if v := cache.Get(id); v != nil { return v, nil }
    log := db.Get(id)
    cache.Set(id, log, 5*time.Minute)
    return log, nil
}
```

Q14

What are the trade-offs between in-process vs distributed cache?

Difficulty:
Medium

	In-Process	Distributed (Redis)
Latency	~0ms (no network)	1-5ms (network hop)
Shared	No (per-instance)	Yes (all instances)
Consistency	Each instance diverges	Single source of truth
Capacity	Limited by JVM/Go heap	Terabytes possible
Eviction	GC pressure	No GC, LRU/LFU
Persistence	Lost on restart	Optional (RDB/AOF)
Complexity	Simple	Connection management
Cost	Free (RAM already there)	Extra infrastructure
Best for	Hot computation, L1 cache	Session state, rate limiting

Recommended pattern: L1 (in-process) + L2 (Redis)

L1: last 1K hot keys, 5s TTL (very short for freshness)

L2: all cached data, minutes/hours TTL

Cache L2 result in L1 on first access

0ms for L1 hits, 1ms for L2 hits, 10ms for DB misses

Q15

What is a session cache and how do you implement it?

Difficulty: Easy

```
Session cache: store user session data in Redis
Key: session:{session_id}
Value: JSON of session data (user_id, roles, preferences)
TTL: 24 hours (or session duration)

Advantages over DB sessions:
Sub-millisecond lookup (every authenticated request)
Easy horizontal scaling (any server reads from Redis)
TTL handles expiry automatically

Implementation:
// Store session
sessionID := uuid.New().String()
sessionData, _ := json.Marshal(Session{UserID: 42, Roles: []string{"admin"}})
redis.Set(ctx, "session:"+sessionID, sessionData, 24*time.Hour)

// Retrieve session
data, err := redis.Get(ctx, "session:"+sessionID).Result()
var session Session
json.Unmarshal([]byte(data), &session)

// Extend on activity
redis.Expire(ctx, "session:"+sessionID, 24*time.Hour)

// Logout
redis.Del(ctx, "session:"+sessionID)

Security:
Session ID = cryptographically random UUID
Store in HttpOnly + Secure cookie
HTTPS only
```

Q16

What is a rate limiter using Redis?

Difficulty:
Medium

Rate limiter: limit requests per user/IP per time window

Sliding window counter (most common):

```
// Token bucket with Redis (atomic via Lua script)
local count = redis.call('INCR', KEYS[1])
if count == 1 then
    redis.call('EXPIRE', KEYS[1], ARGV[1])
end
return count

// Go:
func isAllowed(ctx context.Context, rdb *redis.Client, key string, limit int, window time.Duration) (bool, error) {
    luaScript := redis.NewScript(`
        local count = redis.call('INCR', KEYS[1])
        if count == 1 then redis.call('EXPIRE', KEYS[1], ARGV[1]) end
        return count
    `)
    count, err := luaScript.Run(ctx, rdb, []string{key},
        int(window.Seconds())).Int()
    return count <= limit, err
}

// Usage:
allowed, _ := isAllowed(ctx, rdb, "rate:ip:1.2.3.4", 100, time.Minute)
if !allowed { return http.StatusTooManyRequests }

// Response headers:
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 45
X-RateLimit-Reset: 1609459200
```

Q17

What is a distributed lock using Redis?

Difficulty: Hard

Distributed lock: ensure only one process executes a critical section
 Across multiple machines (unlike `sync.Mutex` which is process-local)

Redis SETNX-based lock:

```
// Acquire: SET key value NX PX ttl_ms (atomic!)
// NX = only set if not exists
// PX = expire in milliseconds
```

```
func acquireLock(ctx context.Context, rdb *redis.Client, resource string, ttl time.Duration) (string, bool) {
    token := uuid.New().String()
    ok, _ := rdb.SetNX(ctx, "lock:"+resource, token, ttl).Result()
    return token, ok
}
```

// Release: Lua script – only delete if WE own it

```
func releaseLock(ctx context.Context, rdb *redis.Client, resource, token string) {
    luaScript := redis.NewScript(`
        if redis.call('GET', KEYS[1]) == ARGV[1] then
            return redis.call('DEL', KEYS[1])
        end
        return 0
    `)
    luaScript.Run(ctx, rdb, []string{"lock:" + resource}, token)
}
```

Redlock (multi-node):

Acquire lock on $N/2+1$ Redis nodes (majority)
 Use when single Redis node failure is unacceptable

Library: github.com/bsm/redislock (recommended)

Q18

What is cache-aside with singleflight in Go?

Difficulty:
 Medium

```

import "golang.org/x/sync/singleflight"

type UserCache struct {
    rdb    *redis.Client
    db     *sql.DB
    group singleflight.Group
}

func (c *UserCache) GetUser(ctx context.Context, id int64) (*User, error) {
    // Check cache first
    key := fmt.Sprintf("user:%d", id)
    data, err := c.rdb.Get(ctx, key).Bytes()
    if err == nil {
        var u User
        return &u, json.Unmarshal(data, &u)
    }
    if !errors.Is(err, redis.Nil) {
        return nil, err
    }

    // Cache miss: singleflight deduplicates concurrent DB requests
    result, err, _ := c.group.Do(key, func() (interface{}, error) {
        var u User
        if err := c.db.QueryRowContext(ctx,
            "SELECT * FROM users WHERE id=$1", id).Scan(&u); err != nil {
            return nil, err
        }
        data, _ := json.Marshal(u)
        // Jittered TTL prevents stampede on popular keys
        ttl := 5*time.Minute + time.Duration(rand.Intn(60))*time.Second
        c.rdb.Set(ctx, key, data, ttl)
        return &u, nil
    })
    if err != nil { return nil, err }
    return result.(*User), nil
}

```

Q19

What is content-addressable caching?

Difficulty:
Medium

Content-addressable: cache key = hash of content/request
Same request parameters → same key → deterministic cache hit

Examples:

key = sha256(sql_query + params) → cache query result
key = sha256(template + data) → cache rendered HTML
key = sha256(image_url + dimensions) → cache resized image

Advantages:

Natural deduplication: identical requests always hit same cache entry
No need to manually construct cache keys
Immutable content: can cache indefinitely (content never changes for same hash)

```
// Go:
import "crypto/sha256"

func queryKey(query string, args ...interface{}) string {
    h := sha256.New()
    fmt.Fprintf(h, "%s:%v", query, args)
    return fmt.Sprintf("query:%x", h.Sum(nil))
}

// CDN: Content-Addressable Storage
// Vite/Webpack: bundle.abc123.js (hash in filename → infinite cache TTL)
// Docker layers: content-addressed → layer shared between images
```

Q20

What are cache metrics you should monitor?

Difficulty: Easy

Key Redis metrics:

```
redis-cli INFO all | grep -E "keyspace|memory|clients|stats"
```

Memory:

- used_memory: total memory used
- used_memory_peak: peak memory
- mem_fragmentation_ratio: >1.5 = fragmentation issue

Performance:

- keyspace_hits: successful lookups
- keyspace_misses: failed lookups
- hit_ratio: hits / (hits + misses) → alert if < 90%
- instantaneous_ops_per_sec: current OPS

Connections:

- connected_clients: active connections
- blocked_clients: clients waiting (BLPOP, etc.)

Eviction:

- evicted_keys: keys removed due to memory limit
- expired_keys: keys expired by TTL

Replication:

- master_repl_offset vs slave_repl_offset → replication lag

Prometheus metrics: redis_exporter

Alert thresholds:

- hit_ratio < 90% → cache not effective
- used_memory > 80% of maxmemory → about to evict
- evicted_keys increasing → memory pressure
- replication lag > 100MB → replica falling behind

2. Redis Data Structures & Internals

Q21

What are Redis data structures and their use cases?

Difficulty: Easy

String: **any** binary-safe value up to 512MB
SET key value [EX seconds] [NX|XX]
INCR key → atomic counter
Use: cache values, counters, sessions, flags

List: doubly-linked list (fast head/tail ops)
LPUSH/RPUSH, LPOP/RPOP, LRange, LLEN
Use: queues (RPUSH+LPOP), recent items, timelines

Hash: field-value **map** within a key
HSET key field value, HGET key field, HGETALL
Use: user profiles, object fields (memory-efficient **for** small objects)

Set: unordered unique elements
SADD, SMEMBERS, SINTER, SUNION, SCARD, SISMEMBER
Use: unique visitors, tags, friends, online users

Sorted Set (ZSet): set ordered by float score
ZADD key score member, ZRange, ZREVRANGE, ZRANK, ZSCORE
Use: leaderboards, rate limiting, delayed queues, timelines

HyperLogLog: approximate cardinality estimation (~0.81% **error**)
PFADD, PFCOUNT, PFMERGE
Use: unique visitor count (1M+ items, tiny memory ~12KB)

Streams: append-only log with consumer groups
XADD, XREAD, XREADGROUP, XACK
Use: event sourcing, message queue with reply tracking

Bitmap: bit operations on strings
SETBIT, GETBIT, BITCOUNT, BITOP
Use: user activity tracking, feature flags per user, bloom filters

Q22

What is Redis Sorted Set and how does it work internally?

Difficulty: Hard

Sorted Set: elements with associated float score, ordered by score
 Unique elements, non-unique scores allowed
 Supports: $O(\log N)$ insert/update, $O(\log N + M)$ range queries

Internal implementation:

Small (≤ 128 elements, ≤ 64 bytes each): ziplist (compact array)

Large: skiplist + hashtable

skiplist: $O(\log N)$ for most operations, $O(1)$ amortized insert

hashtable: $O(1)$ ZSCORE lookup

Operations:

ZADD leaderboard 1500 "alice"	$O(\log N)$
ZSCORE leaderboard "alice"	$O(1)$
ZRANK leaderboard "alice"	$O(\log N)$ (position from low)
ZREVRANK leaderboard "alice"	$O(\log N)$ (position from high)
ZRANGE leaderboard 0 9	$O(\log N + M)$ top 10 by score
ZREVRANGE leaderboard 0 9 WITHSCORES	
ZRANGEBYSCORE lb 1000 2000	$O(\log N + M)$ score range
ZRANGEBYLEX lb "[a" "[z"	$O(\log N + M)$ lexicographic range
ZINCRBY leaderboard 10 "alice"	$O(\log N)$ atomic score increment
ZREM leaderboard "alice"	$O(\log N)$

Use cases:

Leaderboard: ZADD score player $\rightarrow$ ZREVRANGE top10

Rate limiter: sorted set of timestamps per user

Delayed queue: score = execute_at timestamp

Timeline: score = created_at timestamp

Q23

What is the Redis Pub/Sub model?

Difficulty:
Medium

Redis Pub/Sub: fire-and-forget broadcast messaging
No message persistence (missed `if` no subscriber)
No acknowledgment

Commands:

```
SUBSCRIBE channel [channel ...]  
PSUBSCRIBE pattern [pattern ...] (glob patterns)  
PUBLISH channel message  
UNSUBSCRIBE
```

Example (Go):

```
// Subscriber:  
pubsub := rdb.Subscribe(ctx, "order-updates")  
defer pubsub.Close()  
for msg := range pubsub.Channel() {  
    processUpdate(msg.Payload)  
}  
  
// Publisher:  
rdb.Publish(ctx, "order-updates", jsonPayload)
```

Limitations:

- Messages lost `if` subscriber is disconnected
- No message history / replay
- No consumer groups
- Fire-and-forget only

Use cases:

- Real-time notifications (user online/offline)
- Cache invalidation signals (broadcast `"invalidate user:42"`)
- Live dashboard updates

For reliable messaging: use Redis Streams (with XREADGROUP)

Q24

What are Redis Streams?

Difficulty: Hard

Redis Streams (Redis 5.0+): append-only log with consumer groups
 Persistent messages (unlike Pub/Sub)
 Consumer groups (multiple consumers, each gets different messages)
 Acknowledgment support

Commands:

XADD stream * field1 val1 field2 val2	append message
XREAD COUNT 10 STREAMS stream 0	read from beginning
XLEN stream	message count

Consumer groups:

XGROUP CREATE stream group1 \$	create group
XREADGROUP GROUP group1 consumer1 COUNT 10 STREAMS stream >	
XACK stream group1 message_id	acknowledge

// Go example:

```
rdb.XAdd(ctx, &redis.XAddArgs{
    Stream: "orders",
    Values: map[string]interface{}{"order_id": 42, "amount": "99.99"},
})
```

// Consumer group read

```
msgs, _ := rdb.XReadGroup(ctx, &redis.XReadGroupArgs{
    Group:    "processors",
    Consumer: "worker-1",
    Streams:  []string{"orders", ">"},
    Count:    10,
})
for _, msg := range msgs[0].Messages {
    processOrder(msg.Values)
    rdb.XAck(ctx, "orders", "processors", msg.ID)
}
```

vs Kafka: Redis Streams simpler but lower throughput, no partition-level parallelism

Q25

What is Redis persistence (RDB vs AOF)?

Difficulty:
Medium

```
RDB (Redis Database Snapshot):
  Periodic full snapshot to disk (binary format)
  Default: save every 60s if 10K changes (configurable)
  Fast restart (load snapshot in seconds)
  Risk: lose up to 60s of data on crash

# redis.conf
save 900 1    # save after 900s if at least 1 key changed
save 300 10   # save after 300s if at least 10 keys changed
save 60 10000 # save after 60s if at least 10000 keys changed

AOF (Append-Only File):
  Log every write command
  appendfsync: always (safe, slow), everysec (balanced), no (fast, risky)
  Durability: lose at most 1s of data (everysec)
  Larger files, slower restart (replay all commands)

  appendonly yes
  appendfsync everysec # recommended: sync to disk every second

Both (hybrid, recommended):
  Load RDB on startup (fast)
  Use AOF for ongoing durability
  aof-use-rdb-preamble yes # AOF file starts with RDB snapshot

For cache-only (no persistence):
  Comment out all save lines
  appendonly no
  → Fastest, data lost on restart (acceptable for pure cache)
```

Q26

What is Redis memory optimization?

Difficulty: Hard

```
Memory-efficient encodings (automatic):
  Small hash (≤128 fields, ≤64 bytes each) → ziplist (compact)
  Small sorted set → ziplist
  Small set → intset (integers) or listpack
  Small list → listpack

Configure thresholds:
  hash-max-listpack-entries 128
  hash-max-listpack-value 64
  zset-max-listpack-entries 128
  zset-max-listpack-value 64

Memory savings:
  ziplist vs hashtable: 4-10× less memory for small objects
  intset vs hashtable: 2-5× less memory for integer sets

Compression:
  no: no compression (default)
  lz4: compress RDB and AOF

Key naming:
  Short keys: "u:42:sess" vs "user:42:session" → saves memory
  Shared prefix in hash: HSET u:42 name "Alice" vs SET u:42:name "Alice"
  Hash approach: 10× more memory efficient for many small values

Memory analysis:
  redis-cli --bigkeys           # find largest keys
  redis-cli MEMORY USAGE key    # bytes for specific key
  redis-cli OBJECT ENCODING key # current encoding
  redis-cli DEBUG JMAP          # memory breakdown
  redis-cli SCAN 0 MATCH "user:*" COUNT 100 # iterate keys safely
```

Q27

What is Redis pipelining?

Difficulty:
Medium

Pipelining: send multiple commands in one network round-trip
 Reduces RTT overhead dramatically
 Commands batched, sent together, responses received together

Without pipelining:

GET key1 → wait 1ms → GET key2 → wait 1ms → GET key3 → wait 1ms = 3ms total

With pipelining:

[GET key1, GET key2, GET key3] → send once → receive all → 1ms total

```
// Go with go-redis:
pipe := rdb.Pipeline()
get1 := pipe.Get(ctx, "key1")
get2 := pipe.Get(ctx, "key2")
pipe.Set(ctx, "key3", "value", time.Hour)
pipe.Exec(ctx)

val1, _ := get1.Result()
val2, _ := get2.Result()

// TxPipeline: pipeline wrapped in MULTI/EXEC (transaction)
txPipe := rdb.TxPipeline()
// ... commands ...
txPipe.Exec(ctx)
```

When to use:

Reading/writing many keys at once
 Batch operations (populate cache, read multiple users)

When NOT to use:

Commands depend on previous result (need scripting/transactions)
 Very low volume (overhead not worth it)

Q28

What is a Redis Lua script and when do you use it?

Difficulty: Hard

Lua scripts: execute multiple commands atomically on Redis
 Server-side execution: no network round-trips between commands
 Atomic: no other command runs between script commands
 Use: compound operations requiring atomicity

```

// Atomic check-and-set (compare-and-swap):
local current = redis.call('GET', KEYS[1])
if current == ARGV[1] then
    redis.call('SET', KEYS[1], ARGV[2])
    return 1
end
return 0

// Go:
script := redis.NewScript(`
    local current = redis.call('GET', KEYS[1])
    if current == ARGV[1] then
        redis.call('SET', KEYS[1], ARGV[2])
        return 1
    end
    return 0
`)
result, err := script.Run(ctx, rdb, []string{"mykey"}, "expected", "new_value").Int()

// Rate limiter (atomic):
local count = redis.call('INCR', KEYS[1])
if count == 1 then redis.call('EXPIRE', KEYS[1], ARGV[1]) end
return count

// Lock release (only if owner):
if redis.call('GET', KEYS[1]) == ARGV[1] then
    return redis.call('DEL', KEYS[1])
end
return 0

Caching scripts: EVALSHA (cache by SHA1 hash) vs EVAL
SCRIPT LOAD script → returns SHA1 → use EVALSHA sha key args
  
```

Q29

What is Redis MULTI/EXEC (transactions)?

Difficulty:
Medium

MULTI/EXEC: queue commands, execute atomically
 Not a full ACID transaction (no rollback on `error`)
 Isolation: commands execute without interruption
 All-or-nothing execution: `if` EXEC called, all queued commands run

Behavior:

MULTI → enter transaction mode (queue commands)
 command1 → queued (not executed yet)
 command2 → queued
 EXEC → execute all queued commands atomically
 DISCARD → cancel transaction
 WATCH key → optimistic locking (abort `if` key changed)

Redis transaction limitations:

No rollback on runtime errors (command already ran)
 If SET fails mid-transaction: previous SETs not undone
 Not like SQL transactions!

WATCH for optimistic locking:

```
WATCH account:42
value = GET account:42
MULTI
SET account:42 (value - 100)
result = EXEC
if result == nil: WATCH detected change, retry

// Go:
_, err = rdb.TxPipelined(ctx, func(pipe redis.Pipeliner) error {
    pipe.Set(ctx, "key1", "val1", 0)
    pipe.Set(ctx, "key2", "val2", 0)
    return nil
})
// Either both set or neither (on connection failure)
```

Q30

What is Redis single-threaded model?

Difficulty:
Medium

```
Redis: single-threaded command processing (mostly)
  One goroutine handles all commands sequentially
  No locking needed for data structures
  No context switching overhead
```

Why fast despite single-thread:

```
All data in RAM (no disk I/O in critical path)
Simple data structures (O(1) or O(log N))
Minimal memory allocation (pre-allocated buffers)
OS socket I/O using epoll/kqueue (non-blocking)
```

Multi-threading in Redis 6.0+:

```
I/O threads: multiple threads for reading/writing sockets
Command execution: still single-threaded
Result: 2-3x throughput improvement on multi-core servers
```

Implications:

```
Slow commands (KEYS *, SMEMBERS on huge set, SORT) block ALL clients
Avoid O(N) commands on large data sets
Use SCAN instead of KEYS (non-blocking iteration)
Use SSCAN/HSCAN/ZSCAN for large sets/hashtables/zsets
```

Benchmarks:

```
Single Redis node: ~100K-1M simple ops/sec
With Redis 6.0 I/O threads: up to 2M+ ops/sec
```

Q31-Q40: Additional Redis Topics

| Q | Topic |

|---|---|

| Q31 | Redis memory maxmemory and its behavior when limit hit |

| Q32 | Redis key expiry: lazy expiry vs active expiry |

| Q33 | Redis OBJECT ENCODING: ziplist vs skiplist vs listpack |

| Q34 | Redis replication: REPLCONF, PSYNC, partial resync |

| Q35 | Redis Sentinel: automatic failover configuration |

| Q36 | Redis keyspace notifications for expired/modified keys |

| Q37 | Redis SCAN vs KEYS: why KEYS is dangerous in production |

| Q38 | Redis GEO commands: GEOADD, GEODIST, GEOSEARCH |

| Q39 | Redis OBJECT FREQ for LFU eviction tracking |

| Q40 | Redis ACL (Access Control Lists) for security |

3. Eviction Policies & TTL

Q41

What are Redis eviction policies?

Difficulty:
Medium

When maxmemory is reached, Redis uses eviction policy:

go

noeviction (default):

- Return **error** on write commands when full

- Use **for**: queues, lists where data loss is unacceptable

allkeys-lru:

- Evict least recently used key from ALL keys

- Best **for**: pure cache (all keys eligible **for** eviction)

volatile-lru:

- Evict LRU key from keys WITH TTL set

- Use: mixed use **case** (cache + persistent data in same Redis)

allkeys-lfu (Redis 4.0+):

- Evict least frequently used from ALL keys

- Better than LRU when some keys accessed infrequently but recently

volatile-lfu:

- Evict LFU from keys with TTL

allkeys-random:

- Random eviction from all keys

volatile-random:

- Random from keys with TTL

volatile-ttl:

- Evict key with shortest TTL first

- Use: prioritize evicting keys about to expire anyway

Recommended **for** pure cache: allkeys-lru or allkeys-lfu

allkeys-lfu: better when some keys are accessed rarely but recently inserted

Q42

What is LRU and how is it implemented efficiently?

Difficulty: Hard

LRU (Least Recently Used): evict key not accessed **for** longest time

Naive LRU: $O(1)$ lookup (hash **map**) + $O(1)$ access order update (doubly-linked list)

Hash **map**: key $\rightarrow$ node in linked list

Linked list: MRU end ... LRU end

On access: move node to MRU end

On evict: remove from LRU end

Redis approximation (not exact LRU):

Redis **3.0+**: pool of eviction candidates

Sample N random keys (maxmemory-samples, **default** **5**)

Evict least recently used among sample

Approximation is 95%+ accurate vs **true** LRU

Much faster and less memory than **true** LRU

LRU vs LFU:

LRU: good **for** temporal locality (recently used = likely to be reused)

LFU: good **for** frequency-based (popular keys always in cache)

LRU problem: long-running low-frequency scans evict hot keys

LFU problem: **new** popular key starts with frequency=1 (might be evicted!)

LFU solution: decay counter (frequency halved every time period)

// Go: implement $O(1)$ LRU cache

```
import "github.com/hashicorp/golang-lru/v2"
```

```
cache, _ := lru.New[string, User](10000)
```

```
cache.Add("user:42", user)
```

```
if v, ok := cache.Get("user:42"); ok { ... }
```

Q43

What is TTL strategy and jitter?

Difficulty:
Medium

TTL (Time To Live): key automatically deleted after N seconds

Setting TTL:

```
SET key value EX 300           # 300 seconds
SET key value PX 300000        # 300000 milliseconds
SET key value EXAT 1700000000  # Unix timestamp
EXPIRE key 300                 # set TTL on existing key
PERSIST key                    # remove TTL (make permanent)
TTL key                        # seconds remaining (-1=no TTL, -2=doesn't exist)
```

TTL strategy:

Short TTL (seconds): frequently changing data (stock prices, sessions)
 Medium TTL (minutes): user profiles, product catalog
 Long TTL (hours/days): rarely changing data, expensive to compute

Jitter: add randomness to prevent synchronized expiration

Without jitter: all keys set at 3:00 PM expire at 3:05 PM → stampede
 With jitter: TTL = base + rand(0, jitter)

```
// Go:
baseTTL := 5 * time.Minute
jitter := time.Duration(rand.Intn(60)) * time.Second
rdb.Set(ctx, key, value, baseTTL+jitter)
```

Sliding TTL: reset TTL on access (session-style)

On each access: EXPIRE key 30min → extends by 30 min from now
 Stays alive as long as accessed, expires when idle

Q44

What is the difference between EXPIRE and EXPIREAT?

Difficulty: Easy

```
EXPIRE key seconds      : expire in N seconds from now
PEXPIRE key milliseconds : expire in N milliseconds from now
EXPIREAT key timestamp  : expire at Unix timestamp (seconds)
PEXPIREAT key timestamp  : expire at Unix timestamp (milliseconds)
```

Use cases:

```
EXPIRE 3600: cache for 1 hour from now
EXPIREAT (midnight): expire at end of day
```

Atomicity with SET:

```
SET key value EX 3600 ← atomic set + expire
vs
SET key value          ← not atomic (race condition if crash between SET and EXPIRE)
EXPIRE key 3600        ← use SET EX form instead
```

TTL introspection:

```
TTL key   → remaining seconds (-1 = no TTL, -2 = doesn't exist)
PTTL key  → remaining milliseconds
```

Change expiry without changing value:

```
EXPIRE key 7200 → extend to 2 hours from now
EXPIRETIME key  → returns absolute expiry timestamp (Redis 7.0+)
```

Q45

What is cache TTL best practice per data type?

Difficulty:
Medium

TTL guidelines by data volatility:

Immutable data (5 min - 24 hours or no TTL):

Product images, static config, country list
SET product:img:42 url TTL=24h

User-specific data (15 min - 4 hours):

User profile, preferences, permissions
SET user:42:profile json TTL=1h

Computed/aggregated data (5 min - 1 hour):

Leaderboard, stats, analytics summaries
ZADD leaderboard score user → rebuild every 5 min via cron

Real-time data (seconds to minutes):

Feed, notifications, online status
SET user:42:online 1 EX 30 # heartbeat every 15s

Session data (sliding, 30 min - 24 hours):

Reset TTL on each request
EXPIRE session:xyz 3600

Rate limits (per window):

TTL = window duration
SET ratelimit:ip:1.2.3.4:2024010112 0 EX 3600

Never cache:

Financial balances, active transactions, security tokens
Anything requiring strong consistency

Q46-Q50: More Eviction Topics

| Q | Topic |

|---|---|

| Q46 | maxmemory-policy selection guide for different workloads |

| Q47 | volatile-lru vs allkeys-lru: when to use each |

| Q48 | Redis memory fragmentation and how to reduce it |

| Q49 | TTL precision: EXPIRE vs PEXPIRE differences |

| Q50 | Cache size estimation: how much memory do you need? |

4. Cache Patterns & Strategies

Q51

What is the cache-aside pattern in detail?

Difficulty:
Medium

Most common caching pattern:

Read path:

1. App checks cache (GET key)
2. Hit → return value
3. Miss → fetch from DB → SET in cache → return value

Write path (option A: delete-on-write – safer):

1. Write to DB
2. DELETE cache key
3. Next read will repopulate from DB

Write path (option B: update-on-write – riskier):

1. Write to DB
2. SET cache key with new value
- Race condition: concurrent write may set stale value

// Go implementation:

```
func (r *Repo) GetUser(ctx context.Context, id int64) (*User, error) {
    key := fmt.Sprintf("user:%d", id)

    // Check cache
    data, err := r.cache.Get(ctx, key).Bytes()
    if err == nil {
        var u User
        return &u, json.Unmarshal(data, &u)
    }

    // DB fallback
    u, err := r.db.GetUser(ctx, id)
    if err != nil { return nil, err }

    // Populate cache (ignore error – cache is best-effort)
    b, _ := json.Marshal(u)
    r.cache.Set(ctx, key, b, 5*time.Minute+jitter())
    return u, nil
}

func (r *Repo) UpdateUser(ctx context.Context, u *User) error {
    if err := r.db.UpdateUser(ctx, u); err != nil { return err }
    r.cache.Del(ctx, fmt.Sprintf("user:%d", u.ID)) // invalidate
    return nil
}
```

Q52

What is the write-through pattern?

Difficulty:
Medium

Write-through: every write goes to cache AND DB synchronously

1. App writes to cache → cache writes to DB (or app writes both)
2. Both succeed → `return` success
3. Read: always hits cache (never stale after write)

Pros:

Cache always consistent with DB
Read never misses `for` recently written data

Cons:

Write latency: waits `for` both cache and DB
Cache pollution: write data that's never read (wasted memory)
Complex implementation in app or requires cache layer support

// Go:

```
func (r *Repo) CreateProduct(ctx context.Context, p *Product) error {
    // Write to DB first
    if err := r.db.InsertProduct(ctx, p); err != nil { return err }

    // Write to cache
    b, _ := json.Marshal(p)
    key := fmt.Sprintf("product:%d", p.ID)
    r.cache.Set(ctx, key, b, 1*time.Hour)
    return nil
}
```

When to use write-through:

Read-after-write consistency critical (show user their own updates)
Read-heavy with frequent re-reads of written data

When NOT to use:

Write-heavy without read (cache fills with unread data)
High write latency tolerance not available

Q53

What is the write-behind (write-back) pattern?

Difficulty: Hard

Write-behind: write to cache only; async flush to DB
 App writes to cache → returns immediately
 Background process: read from cache → write to DB

Write latency: near 0 (just in-memory write)
 Risk: data loss if cache fails before flush

Implementation:

```
// Write: ultra fast (memory only)
rdb.Set(ctx, key, value, time.Hour)
rdb.LPush(ctx, "write-queue", key) // mark for flush

// Background flusher:
for {
    keys, _ := rdb.BRPop(ctx, time.Second, "write-queue").Result()
    for _, key := range keys[1:] {
        value, _ := rdb.Get(ctx, key).Result()
        db.Upsert(key, value) // flush to DB
    }
}
```

Safer implementation with Redis Streams:

```
XADD write-log * key user:42 op SET value {...}
Background consumer processes stream, writes to DB
```

Use cases:

- Shopping cart (acceptable to lose on crash)
- Game state (saved periodically)
- Analytics counters (approximate is OK)
- Session data (recreatable on next login)

Not for: financial transactions, inventory, anything critical

Q54

What is the refresh-ahead pattern?

Difficulty:
Medium

Refresh-ahead: proactively refresh cache before TTL expires
 Predict what will be needed next → warm up in background
 User never sees cache miss latency

Implementation:

- Option A: background goroutine refreshes popular keys
- Option B: refresh on near-expiry (probabilistic early expiration)

Probabilistic Early Expiration:

When TTL is close to 0, increase probability of background refresh

```
func shouldRefresh(ttl, totalTTL time.Duration, beta float64) bool {
    // beta: controls aggressiveness (typically 1.0)
    // Higher beta = refresh earlier
    elapsed := float64(totalTTL - ttl)
    threshold := -beta * math.Log(rand.Float64()) * float64(totalTTL)
    return elapsed >= float64(totalTTL) - threshold
}

// Simple refresh-ahead with goroutines:
func getWithRefreshAhead(ctx context.Context, key string) (string, error) {
    val, ttl, err := getWithTTL(ctx, key)
    if err != nil { return "", err }

    // If less than 20% TTL remains, refresh in background
    if ttl < originalTTL/5 {
        go func() {
            fresh := fetchFromDB(key)
            rdb.Set(context.Background(), key, fresh, originalTTL+jitter())
        }()
    }
    return val, nil
}
```

Q55

What is stale-while-revalidate?

Difficulty:
Medium

Stale-while-revalidate: serve stale data immediately, refresh async
 Client gets fast response (even if slightly stale)
 Background goroutine fetches fresh data
 Next request gets fresh data

HTTP Cache-Control header:

Cache-Control: max-age=300, stale-while-revalidate=60
 → Fresh for 300s
 → Between 300s-360s: serve stale, trigger background revalidation
 → After 360s: must revalidate (blocking)

Implementation in Go:

```
type cachedValue struct {
    Data      []byte
    ExpiresAt time.Time
    RefreshAt time.Time // start background refresh at this time
}

func get(ctx context.Context, key string) ([]byte, error) {
    v := localCache.Get(key)
    if v == nil { return fetchAndCache(key) } // miss

    if time.Now().After(v.RefreshAt) && time.Now().Before(v.ExpiresAt) {
        go refreshAsync(key) // stale but still valid, refresh in background
    }

    if time.Now().After(v.ExpiresAt) {
        return fetchAndCache(key) // truly expired, blocking fetch
    }

    return v.Data, nil
}
```

Use case: news feed, product listings – slight staleness acceptable

Q56

What is a write-invalidate vs write-update strategy?

Difficulty:
Medium

Write-invalidate (delete on write):

On DB write: DELETE cache key

Next read: miss → fetch from DB → populate cache

Pros: simpler, no race conditions

Cons: first read after write is slower (cold miss)

Write-update (update on write):

On DB write: SET new value in cache

Next read: hit → return fresh cached value

Pros: next read is fast (no cold miss)

Cons: race conditions possible!

Race condition example:

T1: read user (v1) → start UPDATE query

T2: read user (v1) → start UPDATE query → finish → SET cache(v2)

T1: finish → SET cache(v1) ← STALE! T2's update overwritten

Why write-invalidate is safer:

No matter what order writes happen:

After all writes complete: cache is empty or has latest value

Race condition: two deletes for same key → both succeed, no stale data

Best practice: ALWAYS use write-invalidate (delete) not write-update

Exception: truly atomic compare-and-swap with version check

Q57

What is cache-aside with negative caching?

Difficulty:
Medium

Negative caching: cache "not found" results to prevent DB hammering

Problem: request for non-existent key:

user:99999 → cache miss → DB query → not found → cache miss again → DB...

Solution: cache the "not found" result too

```
// Go implementation:
const notFoundMarker = "__NOT_FOUND__"

func getUser(ctx context.Context, id int64) (*User, error) {
    key := fmt.Sprintf("user:%d", id)

    data, err := rdb.Get(ctx, key).Bytes()
    if err == nil {
        if string(data) == notFoundMarker { return nil, ErrNotFound }
        var u User
        return &u, json.Unmarshal(data, &u)
    }

    u, err := db.GetUser(ctx, id)
    if errors.Is(err, ErrNotFound) {
        // Cache "not found" with short TTL (30s – user might be created soon)
        rdb.Set(ctx, key, notFoundMarker, 30*time.Second)
        return nil, ErrNotFound
    }
    if err != nil { return nil, err }

    b, _ := json.Marshal(u)
    rdb.Set(ctx, key, b, 5*time.Minute)
    return u, nil
}

// When user IS created: delete the negative cache entry
func createUser(ctx context.Context, u *User) error {
    if err := db.CreateUser(ctx, u); err != nil { return err }
    rdb.Del(ctx, fmt.Sprintf("user:%d", u.ID)) // clear negative cache
    return nil
}
```

Q58

What is cache stampede prevention with mutex?

Difficulty: Hard

Mutex pattern: only ONE goroutine fetches from DB on miss; others wait

Distributed mutex with Redis:

Lock key: lock:user:42

First requester: acquires lock → fetches from DB → stores in cache → releases lock

Others: wait for lock → lock released → get value from cache

// Go with Redis lock:

```
func getWithMutex(ctx context.Context, key string) (string, error) {
    // Check cache first
    if val, err := rdb.Get(ctx, key).Result(); err == nil { return val, nil }

    // Try to acquire lock
    lockKey := "lock:" + key
    locked, _ := rdb.SetNX(ctx, lockKey, "1", 5*time.Second).Result()

    if !locked {
        // Another goroutine is fetching; wait and retry
        time.Sleep(50 * time.Millisecond)
        return getWithMutex(ctx, key) // retry (will likely hit cache now)
    }
    defer rdb.Del(ctx, lockKey)

    // Double-check cache (another goroutine may have populated it)
    if val, err := rdb.Get(ctx, key).Result(); err == nil { return val, nil }

    // Fetch from DB
    val := fetchFromDB(key)
    rdb.Set(ctx, key, val, 5*time.Minute)
    return val, nil
}

// Better: use golang.org/x/sync/singleflight for in-process dedup
var g singleflight.Group
val, _, _ := g.Do(key, func() (interface{}, error) {
    return fetchFromDB(key)
})
```

Q59

What is cache hierarchies (L1/L2)?

Difficulty:
Medium

Multi-level cache: L1 (in-process) + L2 (distributed Redis)

L1 (in-process):

Location: application memory
 Latency: ~0ms (no network)
 Capacity: small (1K-100K items, limited by heap)
 Scope: per-instance only (not shared)
 TTL: very short (seconds to minutes)

L2 (distributed):

Location: Redis cluster
 Latency: 1-5ms (network)
 Capacity: large (GB to TB)
 Scope: shared across all instances
 TTL: minutes to hours

Algorithm:

Read: check L1 → hit **return**; miss → check L2 → hit: store in L1, **return**
 → L2 miss: fetch DB → store in L1 + L2 → **return**
 Write: invalidate L1 + **delete** L2 key

// Go L1 implementation:

```
import lru "github.com/hashicorp/golang-lru/v2"
```

```
type MultiLevelCache struct {
```

```
    l1 *lru.Cache[string, []byte] // in-process
    l2 *redis.Client              // Redis
```

```
}
```

```
func (c *MultiLevelCache) Get(ctx context.Context, key string) ([]byte, error) {
    if v, ok := c.l1.Get(key); ok { return v, nil } // L1 hit
```

```
    v, err := c.l2.Get(ctx, key).Bytes() // L2 hit
    if err == nil {
        c.l1.Add(key, v) // populate L1
        return v, nil
    }
```

```
    return nil, err // total miss
```

```
}
```

Q60

What is a CDN cache and how does it differ from Redis?

Difficulty:
Medium

CDN (Content Delivery Network) cache:
 Geographically distributed edge servers
 Cache HTTP responses at network edge
 Serves users from nearest PoP (Point of Presence)

Latency: 5-20ms (user to CDN PoP, not origin)
 Capacity: petabytes across all PoPs
 Use: static assets (images, JS, CSS), public API responses

Redis cache:
 Single or clustered in-datacenter
 Application-level caching
 Complex data structures

Latency: 1-5ms (within datacenter)
 Capacity: TB in cluster
 Use: session state, rate limiting, DB query results

Cache-Control headers for CDN:

public, max-age=3600	→ cache for 1 hour at CDN + browser
public, s-maxage=3600	→ cache 1h at CDN, browser uses max-age
no-cache	→ always revalidate (but can serve stale)
no-store	→ never cache (sensitive data)
stale-while-revalidate=60	→ serve stale, refresh async

CDN invalidation:

Purge by URL: `curl -X DELETE https://cdn/purge?url=...`
 Purge by tag: all assets tagged "product:42"
 Versioned URLs: /static/bundle.abc123.js (no purge needed)

Q61-Q65: More Cache Patterns

| Q | Topic |

|---|---|

| Q61 | Caching SQL query results: when and how |

| Q62 | Function-level memoization in Go |

| Q63 | Cache-aside with circuit breaker for Redis failures |

| Q64 | Idempotency key caching for API deduplication |

| Q65 | Batch cache operations for N+1 prevention |

5. Distributed Caching & Redis Cluster

Q66

What is Redis Cluster and how does it shard data?

Difficulty: Hard

Redis Cluster: horizontal sharding across multiple Redis nodes

Total hash slots: 16,384

Each key $\rightarrow$ $\text{CRC16}(\text{key}) \% 16384 \rightarrow$ hash slot $\rightarrow$ node

Node assignment:

3 primary nodes: Node A (slots 0-5460), B (5461-10922), C (10923-16383)

Each primary has 1-2 replicas for HA

Minimum: 3 primaries + 3 replicas = 6 nodes

Hash tags: {user_id} $\rightarrow$ hash only the part in {}

{user:42}:cart and {user:42}:profile $\rightarrow$ same slot

Enables MULTI/EXEC across multiple keys (same slot only)

Scaling:

Add nodes: Redis Cluster rebalances slots automatically

Remove nodes: drain slots first, then remove

Limitations:

Cross-slot operations: must use hash tags or pipeline separately

Lua scripts: all keys must be in same slot

Pub/Sub: works cluster-wide

Client routing:

Client receives MOVED error if queries wrong node $\rightarrow$ updates routing table

Smart clients (redis-go Cluster, ioredis): route automatically

When to use:

Dataset > 100GB OR ops/sec > 1M (single node limit)

Single node: ~100GB RAM, ~1M ops/sec

Cluster: horizontal scale to any size

Q67

What is Redis Sentinel?

Difficulty:
Medium

Redis Sentinel: HA for single-master Redis setup (no sharding)
 Monitors primary + replicas
 Automatic failover if primary dies
 Service discovery: clients ask Sentinel "who is primary?"

Architecture:

- 3 Sentinel processes (quorum=2 for decisions)
- 1 Primary + N replicas

Failover process:

1. Sentinels detect primary unreachable (3× ping timeout)
2. Quorum (2/3) agrees primary is down (SDOWN → ODOWN)
3. Leader Sentinel elected (Raft-like)
4. Leader promotes best replica to primary
5. Reconfigures other replicas to follow new primary
6. Notifies clients via Sentinel API

Client connection:

Connect to Sentinel → ask for primary address → connect to primary
 On failover: Sentinel notifies → client reconnects to new primary

// Go:

```
rdb := redis.NewFailoverClient(&redis.FailoverOptions{
    MasterName: "mymaster",
    SentinelAddr: []string{"sentinel1:26379", "sentinel2:26379", "sentinel3:26379"},
})
```

Sentinel vs Cluster:

Sentinel: HA only, single master, no sharding
 Cluster: HA + sharding, multiple masters
 Use Sentinel when: data fits single node, want simpler ops
 Use Cluster when: need horizontal scale

Q68

What is consistent hashing for cache distribution?

Difficulty: Hard

Consistent hashing: distribute keys across N cache nodes

Minimizes remapping when nodes are added/removed

Problem with naive hashing:

key $\rightarrow$ hash(key) % N $\rightarrow$ server

Add 1 server (N=4 $\rightarrow$ 5): almost ALL keys remap $\rightarrow$ massive cache misses

Consistent hashing:

Hash space: 0 $\rightarrow$ 2^{32} (ring)

Servers placed at positions on ring (via hash(server_name))

Key maps to nearest server clockwise on ring

Add server: only keys between (new, predecessor) remapped

Remove server: only keys on that server remap to successor

Impact: K/N keys remapped (K=total keys, N=nodes)

Virtual nodes:

Each real node = 150 virtual nodes on ring

$\rightarrow$ Better distribution, more even load

$\rightarrow$ New server takes equal share from all existing servers

// Go implementation:

```
import "github.com/stathat/consistent"
```

```
c := consistent.New()
```

```
c.Add("server1")
```

```
c.Add("server2")
```

```
c.Add("server3")
```

```
server, _ := c.Get("user:42") // consistently routes to same server
```

Used by: Memcached clients, Redis clients (some), CDN, service discovery

Q69

What is a cache hot key and how do you solve it?

Difficulty: Hard

Hot key: one cache key receiving disproportionate traffic
 Example: celebrity profile read by 1M followers/sec
 Redis: single-threaded → one key becomes bottleneck

Solutions:

1. Local in-process cache (L1):

Each app server caches the hot key locally

TTL = 5-10 seconds (short for freshness)

100 app servers × 10K req/sec → 1M req/sec served without Redis

```
var localCache = sync.Map{}
```

```
func getHotKey(ctx context.Context, key string) (string, error) {
    if v, ok := localCache.Load(key); ok {
        return v.(string), nil // 0ms hit
    }
    val, err := rdb.Get(ctx, key).Result() // Redis fallback
    if err == nil {
        localCache.Store(key, val)
        time.AfterFunc(5*time.Second, func() { localCache.Delete(key) })
    }
    return val, err
}
```

2. Key replication / scatter reads:

key:copy1, key:copy2, key:copy3 on different Redis nodes

Read: key:"copy" + strconv.Itoa(rand.Intn(3)+1)

Write: update all copies

3. Redis Cluster + hash tags:

Distribute natural sharding key better

4. Read replicas:

Redis replica nodes for read-only traffic

Q70

What is Redis cross-slot operations and hash tags?

Difficulty: Hard

Redis Cluster limitation: multi-key operations only for keys in same slot

MGET key1 key2 → keys must be in same slot
 MULTI/EXEC transactions → all keys same slot
 Lua scripts → all keys same slot
 SUNIONSTORE, ZINTERSTORE → same slot

Hash tags: {tag} forces key into hash slot of tag
 CRC16 computed on tag only, not full key

{user:42}:cart → slot of "user:42"
 {user:42}:profile → slot of "user:42" ← SAME slot!
 {user:42}:sessions → slot of "user:42" ← SAME slot!

Now: MGET {user:42}:cart {user:42}:profile → works!
 Now: MULTI/EXEC across these keys → works!

Downside: all keys with same tag → same slot → hot spot
 Don't tag everything with same value
 Tag by entity that needs atomic operations

```
// Go:
keys := []string{
    fmt.Sprintf("{user:%d}:cart", userID),
    fmt.Sprintf("{user:%d}:profile", userID),
}
vals, _ := rdb.MGet(ctx, keys...).Result()
```

Redis Cluster routing:

MOVED 3999 127.0.0.1:6381 → wrong slot, go to this node
 ASK 3999 127.0.0.1:6381 → slot being migrated, try this node temporarily

Q71-Q80: More Distributed Cache Topics

| Q | Topic |

|---|---|

| Q71 | Memcached vs Redis: when to use each |

| Q72 | Redis read replicas for read scaling |

| Q73 | Cache partition strategies: by user, by feature, by data type |

| Q74 | Geo-distributed caching: CDN + regional Redis |

| Q75 | Cache warm-up strategies for Blue-Green deployments |

| Q76 | Redis connection pool tuning for high-concurrency Go apps |

| Q77 | Cache key design: namespacing, versioning, structure |

| Q78 | Cache serialization: JSON vs MessagePack vs Protobuf |

| Q79 | Redis cluster topology: primary-only vs primary-replica per shard |

| Q80 | Handling Redis cluster failover in Go applications |

6. Advanced Topics & Go Integration

Q81

How do you integrate Redis with Go using go-redis?

Difficulty: Easy

```
import "github.com/redis/go-redis/v9"

// Single node
rdb := redis.NewClient(&redis.Options{
    Addr:      "localhost:6379",
    Password:  "",
    DB:        0,
    PoolSize:  10,
    MinIdleConns: 5,
    MaxRetries: 3,
    DialTimeout: 5 * time.Second,
    ReadTimeout: 3 * time.Second,
    WriteTimeout: 3 * time.Second,
})

// Sentinel (HA)
rdb = redis.NewFailoverClient(&redis.FailoverOptions{
    MasterName:  "mymaster",
    SentinelAddr: []string{"s1:26379", "s2:26379", "s3:26379"},
    Password:    "password",
    PoolSize:    10,
})

// Cluster
rdb = redis.NewClusterClient(&redis.ClusterOptions{
    Addrs: []string{"node1:6379", "node2:6379", "node3:6379"},
    PoolSize: 5,
})

// Ping
if err := rdb.Ping(ctx).Err(); err != nil {
    log.Fatalf("redis connection failed: %v", err)
}
```

Q82

How do you implement a leaderboard in Go with Redis?

Difficulty:
Medium


```

type LeaderboardService struct { rdb *redis.Client }

const leaderboardKey = "leaderboard:global"

func (s *LeaderboardService) AddScore(ctx context.Context, playerID string, score float64) error {
    return s.rdb.ZAdd(ctx, leaderboardKey, redis.Z{
        Score: score,
        Member: playerID,
    }).Err()
}

func (s *LeaderboardService) IncrementScore(ctx context.Context, playerID string, delta float64) (float64, error) {
    return s.rdb.ZIncrBy(ctx, leaderboardKey, delta, playerID).Result()
}

func (s *LeaderboardService) GetTopN(ctx context.Context, n int) ([]redis.Z, error) {
    return s.rdb.ZRevRangeWithScores(ctx, leaderboardKey, 0, int64(n-1)).Result()
}

func (s *LeaderboardService) GetRank(ctx context.Context, playerID string) (int64, error) {
    rank, err := s.rdb.ZRevRank(ctx, leaderboardKey, playerID).Result()
    return rank + 1, err // 1-indexed
}

func (s *LeaderboardService) GetNeighbors(ctx context.Context, playerID string, spread int) ([]redis.Z, error) {
    rank, _ := s.rdb.ZRevRank(ctx, leaderboardKey, playerID).Result()
    start := rank - int64(spread)
    if start < 0 { start = 0 }
    return s.rdb.ZRevRangeWithScores(ctx, leaderboardKey, start, rank+int64(spread)).Result()
}

```

Q83

How do you implement Redis-based caching middleware in Go?

Difficulty:
Medium

```
// HTTP middleware for response caching
func CacheMiddleware(rdb *redis.Client, ttl time.Duration) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            // Only cache GET requests
            if r.Method != http.MethodGet {
                next.ServeHTTP(w, r)
                return
            }

            key := "http:" + r.URL.RequestURI()

            // Check cache
            if data, err := rdb.Get(r.Context(), key).Bytes(); err == nil {
                w.Header().Set("X-Cache", "HIT")
                w.Header().Set("Content-Type", "application/json")
                w.Write(data)
                return
            }

            // Capture response
            rec := &responseRecorder{ResponseWriter: w, body: &bytes.Buffer{}}
            next.ServeHTTP(rec, r)

            // Cache successful responses
            if rec.status == http.StatusOK {
                rdb.Set(r.Context(), key, rec.body.Bytes(),
                    ttl+time.Duration(rand.Intn(30))*time.Second)
                w.Header().Set("X-Cache", "MISS")
            }
        })
    }
}

type responseRecorder struct {
    http.ResponseWriter
    status int
    body   *bytes.Buffer
}

func (r *responseRecorder) WriteHeader(status int) {
    r.status = status; r.ResponseWriter.WriteHeader(status)
}

func (r *responseRecorder) Write(b []byte) (int, error) {
    r.body.Write(b); return r.ResponseWriter.Write(b)
}
```

Q84

How do you implement a Redis-based job queue in Go?

Difficulty: Hard

```
// Simple reliable job queue using Redis List + processing set
type Queue struct {
    rdb      *redis.Client
    queueKey string
    processingKey string
}

func (q *Queue) Enqueue(ctx context.Context, payload []byte) error {
    return q.rdb.LPush(ctx, q.queueKey, payload).Err()
}

func (q *Queue) Dequeue(ctx context.Context, timeout time.Duration) ([]byte, error) {
    // BRPOPLPUSH: atomically move from queue to processing set
    result, err := q.rdb.BRPopLPush(ctx, q.queueKey, q.processingKey, timeout).Result()
    if err == redis.Nil { return nil, nil } // timeout
    return []byte(result), err
}

func (q *Queue) Acknowledge(ctx context.Context, payload []byte) error {
    // Remove from processing set after successful handling
    return q.rdb.LRem(ctx, q.processingKey, 1, payload).Err()
}

func (q *Queue) RecoverStuck(ctx context.Context, timeout time.Duration) error {
    // Move stuck jobs back to main queue (ran by cron or background worker)
    items, _ := q.rdb.LRange(ctx, q.processingKey, 0, -1).Result()
    for _, item := range items {
        // In production: check timestamp embedded in job to determine age
        q.rdb.RPush(ctx, q.queueKey, item)
        q.rdb.LRem(ctx, q.processingKey, 1, item)
    }
    return nil
}
```

Q85

How do you implement Redis pub/sub for cache invalidation in Go?

Difficulty:
Medium

```
// Multi-server cache invalidation via Redis Pub/Sub

type CacheInvalidator struct {
    rdb      *redis.Client
    localCache sync.Map
}

// Subscriber: listen for invalidation events
func (c *CacheInvalidator) Subscribe(ctx context.Context) {
    pubsub := c.rdb.Subscribe(ctx, "cache:invalidate")
    defer pubsub.Close()

    for {
        select {
        case <-ctx.Done(): return
        case msg := <-pubsub.Channel():
            c.localCache.Delete(msg.Payload)
            log.Printf("invalidated local cache key: %s", msg.Payload)
        }
    }
}

// Publisher: broadcast invalidation when data changes
func (c *CacheInvalidator) InvalidateAll(ctx context.Context, key string) error {
    // 1. Delete from Redis
    c.rdb.Del(ctx, key)
    // 2. Invalidate local cache on this server
    c.localCache.Delete(key)
    // 3. Notify all other servers to invalidate their local caches
    return c.rdb.Publish(ctx, "cache:invalidate", key).Err()
}

// Usage:
inv := &CacheInvalidator{rdb: rdb}
go inv.Subscribe(ctx) // start listener in background

// On data update:
db.UpdateUser(user)
inv.InvalidateAll(ctx, fmt.Sprintf("user:%d", user.ID))
```

Q86

How do you monitor Redis health in Go?

Difficulty: Easy

```

type RedisHealthChecker struct { rdb *redis.Client }

func (h *RedisHealthChecker) Check(ctx context.Context) error {
    // Ping
    if err := h.rdb.Ping(ctx).Err(); err != nil {
        return fmt.Errorf("redis ping failed: %w", err)
    }
    return nil
}

// Readiness probe endpoint
http.HandleFunc("/ready", func(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
    defer cancel()
    if err := checker.Check(ctx); err != nil {
        w.WriteHeader(http.StatusServiceUnavailable)
        json.NewEncoder(w).Encode(map[string]string{"redis": err.Error()})
        return
    }
    json.NewEncoder(w).Encode(map[string]string{"status": "ok"})
})

// Connection pool stats
stats := rdb.PoolStats()
log.Printf("redis pool: hits=%d misses=%d timeouts=%d totalConns=%d idleConns=%d",
    stats.Hits, stats.Misses, stats.Timeouts,
    stats.TotalConns, stats.IdleConns)

// Export to Prometheus:
redisPoolHits.Set(float64(stats.Hits))
redisPoolMisses.Set(float64(stats.Misses))

```

Q87

How do you implement rate limiting with Redis in Go?

Difficulty:
Medium

```

type RateLimiter struct {
    rdb    *redis.Client
    script *redis.Script
}

func NewRateLimiter(rdb *redis.Client) *RateLimiter {
    // Atomic Lua: increment + set expiry if new key
    script := redis.NewScript(`
        local count = redis.call('INCR', KEYS[1])
        if count == 1 then
            redis.call('EXPIRE', KEYS[1], ARGV[1])
        end
        return count
    `)
    return &RateLimiter{rdb: rdb, script: script}
}

type RateLimitResult struct {
    Allowed    bool
    Count      int64
    Remaining  int64
    ResetAt    time.Time
}

func (r *RateLimiter) Allow(ctx context.Context, key string, limit int64, window time.Duration) (*RateLimitResult, error) {
    windowSecs := int(window.Seconds())
    now := time.Now()

    count, err := r.script.Run(ctx, r.rdb,
        []string{fmt.Sprintf("rl:%s:%d", key, now.Unix()/int64(windowSecs))},
        windowSecs,
    ).Int64()
    if err != nil { return nil, err }

    resetAt := time.Unix((now.Unix()/int64(windowSecs)+1)*int64(windowSecs), 0)
    return &RateLimitResult{
        Allowed:    count <= limit,
        Count:      count,
        Remaining:  max(0, limit-count),
        ResetAt:    resetAt,
    }, nil
}

// HTTP middleware:
func RateLimitMiddleware(limiter *RateLimiter) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            result, _ := limiter.Allow(r.Context(), r.RemoteAddr, 100, time.Minute)
            w.Header().Set("X-RateLimit-Remaining", strconv.FormatInt(result.Remaining, 10))
            if !result.Allowed {
                w.Header().Set("Retry-After", strconv.Itoa(int(time.Until(result.ResetAt).Seconds)))
                w.WriteHeader(http.StatusTooManyRequests)
                return
            }
        })
    }
}

```

```
    }  
    next.ServeHTTP(w, r)  
  })  
}  
}
```

go

Q88

How do you implement cache warming on startup in Go?

Difficulty:
Medium

```

type CacheWarmer struct {
    rdb *redis.Client
    db  *pgxpool.Pool
}

func (w *CacheWarmer) WarmProducts(ctx context.Context) error {
    log.Println("warming product cache...")

    rows, err := w.db.Query(ctx,
        "SELECT id, data FROM products ORDER BY views DESC LIMIT 10000")
    if err != nil { return err }
    defer rows.Close()

    // Use pipeline for batch SET
    pipe := w.rdb.Pipeline()
    count := 0
    for rows.Next() {
        var id int64
        var data []byte
        rows.Scan(&id, &data)

        key := fmt.Sprintf("product:%d", id)
        ttl := 30*time.Minute + time.Duration(rand.Intn(300))*time.Second
        pipe.Set(ctx, key, data, ttl)
        count++

        // Execute in batches of 500
        if count%500 == 0 {
            pipe.Exec(ctx)
            pipe = w.rdb.Pipeline()
        }
    }
    if count%500 != 0 { pipe.Exec(ctx) }

    log.Printf("warmed %d products", count)
    return nil
}

// Call on startup before accepting traffic
func main() {
    warmer := &CacheWarmer{rdb: rdb, db: db}
    if err := warmer.WarmProducts(ctx); err != nil {
        log.Printf("cache warm failed (non-fatal): %v", err)
    }
    // Start server...
}

```

Q89

What is a circuit breaker for Redis in Go?

Difficulty: Hard


```

import "github.com/sony/gobreaker"

type ResilientCache struct {
    rdb      *redis.Client
    breaker *gobreaker.CircuitBreaker
}

func NewResilientCache(rdb *redis.Client) *ResilientCache {
    cb := gobreaker.NewCircuitBreaker(gobreaker.Settings{
        Name:      "redis",
        MaxRequests: 5,
        Interval:   30 * time.Second,
        Timeout:    60 * time.Second,
        ReadyToTrip: func(counts gobreaker.Counts) bool {
            return counts.ConsecutiveFailures > 10
        },
        OnStateChange: func(name string, from, to gobreaker.State) {
            log.Printf("Redis circuit breaker: %s → %s", from, to)
        },
    })
    return &ResilientCache{rdb: rdb, breaker: cb}
}

func (c *ResilientCache) Get(ctx context.Context, key string) ([]byte, error) {
    result, err := c.breaker.Execute(func() (interface{}, error) {
        return c.rdb.Get(ctx, key).Bytes()
    })
    if err != nil {
        if errors.Is(err, gobreaker.ErrOpenState) {
            return nil, nil // cache unavailable, fall through to DB
        }
        return nil, err
    }
    return result.([]byte), nil
}

func (c *ResilientCache) Set(ctx context.Context, key string, value []byte, ttl time.Duration) {
    c.breaker.Execute(func() (interface{}, error) {
        return nil, c.rdb.Set(ctx, key, value, ttl).Err()
    })
    // Ignore cache set errors (best-effort)
}

```

Q90

How do you implement idempotency keys with Redis in Go?

Difficulty:
Medium

```
// Idempotency: same request with same key → same result (no duplicate processing)

type IdempotencyStore struct{ rdb *redis.Client }

const idempotencyTTL = 24 * time.Hour

type StoredResult struct {
    StatusCode int    `json:"status_code"`
    Body       []byte `json:"body"`
    CreatedAt  int64  `json:"created_at"`
}

func (s *IdempotencyStore) GetOrProcess(
    ctx context.Context,
    key string,
    handler func() (int, []byte, error),
) (int, []byte, error) {
    redisKey := "idempotency:" + key

    // Check if already processed
    data, err := s.rdb.Get(ctx, redisKey).Bytes()
    if err == nil {
        var result StoredResult
        json.Unmarshal(data, &result)
        return result.StatusCode, result.Body, nil // return cached result
    }

    // Not processed: use singleflight to prevent concurrent processing
    var g singleflight.Group
    v, err, _ := g.Do(key, func() (interface{}, error) {
        status, body, err := handler()
        if err != nil { return nil, err }

        result := StoredResult{StatusCode: status, Body: body, CreatedAt: time.Now().Unix()}
        d, _ := json.Marshal(result)
        s.rdb.Set(ctx, redisKey, d, idempotencyTTL)
        return &result, nil
    })
    if err != nil { return 0, nil, err }
    r := v.(*StoredResult)
    return r.StatusCode, r.Body, nil
}

// Usage in payment handler:
status, body, err := idempotency.GetOrProcess(ctx, idempotencyKey, func() (int, []byte, error) {
    result, err := paymentService.Charge(order)
    // ... process payment ...
})
```

Q91-Q100: Additional Advanced Cache Topics

| Q | Topic |

|---|---|

| Q91 | Redis WATCH for optimistic locking in Go |

- | Q92 | Implementing a distributed counter with Redis in Go |
- | Q93 | Cache key versioning for zero-downtime deploys |
- | Q94 | Redis keyspace notifications for event-driven cache expiry |
- | Q95 | Testing Go code that uses Redis (testcontainers, miniredis) |
- | Q96 | Redis memory profiling and optimization techniques |
- | Q97 | Implementing a feature flag service with Redis in Go |
- | Q98 | Cache stampede prevention: Go singleflight vs Redis lock |
- | Q99 | Redis Streams as a reliable job queue vs List-based queue |
- | Q100 | Cache design review: choosing TTL, eviction, and consistency |

Master these 100 questions and you'll handle any caching interview at SDE2 level. Key areas: cache patterns (aside, through, behind), Redis data structures (Sorted Set for leaderboards, Streams for queues), stampede prevention (singleflight + jitter), distributed cache (Cluster, Sentinel, consistent hashing), and Go integration (go-redis, pipeline, Lua scripts). ■

Additional Questions (Q66–Q150)

Q66

What is Redis Cluster and how does it work?

Difficulty: Hard

Redis Cluster: horizontal scaling via sharding across 16384 hash slots.

```
16384 hash slots distributed across N master nodes
Each node handles a range of slots (e.g., node1: 0-5460)

Key → CRC16(key) % 16384 → slot → node

3-node cluster:
Node A: slots 0-5460      (+ replica A')
Node B: slots 5461-10922 (+ replica B')
Node C: slots 10923-16383 (+ replica C')

Replication: each master has 1+ replicas
Failover: replica promotes if master unreachable (majority vote)
```

go

```
# Create cluster
redis-cli --cluster create \
  127.0.0.1:7000 127.0.0.1:7001 127.0.0.1:7002 \
  127.0.0.1:7003 127.0.0.1:7004 127.0.0.1:7005 \
  --cluster-replicas 1

# Check cluster status
redis-cli -p 7000 cluster info
redis-cli -p 7000 cluster nodes
```

bash

```
// Go client (redis/go-redis v9)
rdb := redis.NewClusterClient(&redis.ClusterOptions{
  Addrs: []string{"7000", "7001", "7002"},
  Password: "secret",
})
```

go

Interview tip: "Hash tags {user} force related keys to same slot. Use for multi-key operations (MGET, pipelining). Without same slot: CROSSSLOT error."

Q67

What are hash tags in Redis Cluster?

Difficulty:
Medium

```
Hash tag: {key} – only the part inside {} is hashed for slot assignment
Used to colocate related keys on same node
```

go

Without hash tag:

```
user:1:profile → slot 123 (Node A)
user:1:orders  → slot 456 (Node B)
→ can't MGET both!
```

With hash tag:

```
{user:1}:profile → CRC16("user:1") % 16384 → same slot
{user:1}:orders  → CRC16("user:1") % 16384 → same slot
→ MGET works! Both on same node
```

```
// Ensure keys are on same slot for pipeline
pipe := rdb.Pipeline()
pipe.Get(ctx, "{user:1}:profile")
pipe.Get(ctx, "{user:1}:orders")
pipe.Get(ctx, "{user:1}:cart")
cmds, err := pipe.Exec(ctx)
```

go

Q68

What is Redis Sentinel?

Difficulty:
Medium

```
Redis Sentinel: high availability for single Redis master (not cluster)
Monitors master + replicas, auto-failover if master dies
```

go

Sentinel topology:

```
Sentinel1, Sentinel2, Sentinel3 (need odd number for quorum)
→ Monitor: Redis-Master + Redis-Replica1 + Redis-Replica2
```

Failover process:

1. Sentinels detect master unreachable (down-after-milliseconds)
2. Quorum reached (majority of sentinels agree)
3. One sentinel elected as leader
4. Leader selects best replica (most recent data)
5. Replica promoted to master
6. Other replicas reconfigured to follow new master
7. Clients notified (get new master address)

sentinel.conf:

```
sentinel monitor mymaster 127.0.0.1 6379 2 # quorum=2
sentinel down-after-milliseconds mymaster 5000
sentinel failover-timeout mymaster 60000
```

```
rdb := redis.NewFailoverClient(&redis.FailoverOptions{
    MasterName:    "mymaster",
    SentinelAddrs: []string{":26379", ":26380", ":26381"},
    Password:     "secret",
})
```

go

Q69

What is Redis persistence: RDB vs AOF?

Difficulty: Hard

```
RDB (Redis Database Snapshot):
  Periodic point-in-time snapshot of all data
  Fork process writes snapshot to disk
  Fast restart (loads compact binary)
  Data loss: up to last snapshot interval

CONFIG: save 900 1    # save if 1 key changed in 900s
        save 300 10   # save if 10 keys changed in 300s
        save 60 10000 # save if 10000 keys changed in 60s
FILE: dump.rdb

AOF (Append-Only File):
  Logs every write command
  More durable (configurable fsync policy)
  Larger file, slower restart

appendonly yes
appendfsync everysec # fsync every second (good balance)
# appendfsync always # fsync every write (safest, slowest)
# appendfsync no     # OS decides (fastest, least safe)
FILE: appendonly.aof

Hybrid (Redis 4.0+):
  RDB snapshot + AOF tail (since last snapshot)
  aof-use-rdb-preamble yes
  Best of both: fast load + durability
```

Q70

What is Redis pub/sub?

Difficulty:
Medium

```
// Redis Pub/Sub: simple publish-subscribe (no persistence, no ACK)
// Messages lost if no subscribers
// Not for reliable messaging (use Streams for that)

// Publisher
err := rdb.Publish(ctx, "notifications:user:42", payload).Err()

// Subscriber
pubsub := rdb.Subscribe(ctx, "notifications:user:42")
defer pubsub.Close()

ch := pubsub.Channel()
for msg := range ch {
    fmt.Printf("channel=%s payload=%s\n", msg.Channel, msg.Payload)
}

// Pattern subscribe (wildcard)
pubsub := rdb.PSubscribe(ctx, "notifications:*")
for msg := range pubsub.Channel() {
    fmt.Println(msg.Channel, msg.Payload)
}

// Pub/Sub vs Redis Streams:
// Pub/Sub: fire-and-forget, no persistence, fan-out
// Streams: persistent, consumer groups, replay, ACK
// Use Streams for: task queues, event sourcing, reliable messaging
```

Q71

What are Redis Streams?

Difficulty: Hard

```

// Redis Streams: persistent, ordered log (like Kafka-lite)
// Features: consumer groups, ACK, pending entries, trim

// Produce
id, err := rdb.XAdd(ctx, &redis.XAddArgs{
    Stream: "orders",
    Values: map[string]interface{}{
        "order_id": "42",
        "amount":   "99.99",
        "status":   "pending",
    },
    MaxLen: 10000, // keep last 10K messages
}).Result()
// id = "1234567890-0" (timestamp-sequence)

// Consume (consumer group for load balancing)
rdb.XGroupCreate(ctx, "orders", "order-processors", "0")

msgs, err := rdb.XReadGroup(ctx, &redis.XReadGroupArgs{
    Group:    "order-processors",
    Consumer: "worker-1",
    Streams:  []string{"orders", ">"}, // ">" = new messages only
    Count:    10,
    Block:    5 * time.Second,
}).Result()

for _, msg := range msgs[0].Messages {
    process(msg.Values)
    rdb.XAck(ctx, "orders", "order-processors", msg.ID) // acknowledge
}

// Check pending (unacked) messages
pending, _ := rdb.XPendingExt(ctx, &redis.XPendingExtArgs{
    Stream: "orders", Group: "order-processors",
    Start: "-", Stop: "+", Count: 100,
}).Result()

```

Q72

What are Redis Bloom Filters?

Difficulty: Hard

Bloom Filter: probabilistic data structure
 Test if element is in a set: "possibly yes" OR "definitely no"
 False positives possible, false negatives impossible
 Space-efficient: fixed size regardless of element count

Redis Bloom Filter (RedisBloom module):

```
BF.ADD key item      # add item
BF.EXISTS key item    # check membership
BF.MADD key i1 i2 i3 # batch add
BF.MEXISTS key i1 i2 # batch check
```

Use cases:

- Cache warming: check if item might be in cache before DB query
- Duplicate request detection (has this webhook been processed?)
- Spam email filter
- Username availability check

go-redis with Bloom:

```
rdb.BFAdd(ctx, "processed-events", eventID)
exists, err := rdb.BFExists(ctx, "processed-events", eventID)
```

False positive rate: $p = (1 - e^{(-kn/m)})^k$

k = number of hash functions

n = number of elements

m = bit array size

Lower p = larger bit array needed

Q73

What is Redis HyperLogLog?

Difficulty:
Medium

```
// HyperLogLog: approximate count of unique elements
// Uses ~12KB regardless of cardinality
// Error rate: ~0.81%

// Count unique page views
rdb.PFAdd(ctx, "visitors:2024-01-15", "user1", "user2", "user3")
rdb.PFAdd(ctx, "visitors:2024-01-15", "user1", "user4") // user1 already counted

count, _ := rdb.PFCount(ctx, "visitors:2024-01-15").Result()
// count ≈ 4 (unique visitors)

// Merge multiple HLL (union)
rdb.PFMerge(ctx, "visitors:week",
    "visitors:2024-01-13",
    "visitors:2024-01-14",
    "visitors:2024-01-15",
)
weekCount, _ := rdb.PFCount(ctx, "visitors:week").Result()

// vs exact count with SET:
// SET: exact but O(n) memory (all IDs stored)
// HLL: ~0.81% error, O(1) memory (12KB always)

// Use for: analytics, cardinality estimation
// Not for: when you need exact count or to retrieve items
```

Q74

What is Redis sorted set use cases?

Difficulty:
Medium

```
// Sorted Set (ZSET): members with float scores, sorted by score
// O(log N) for most operations

// Leaderboard
rdb.ZAdd(ctx, "game:leaderboard",
    redis.Z{Score: 9500, Member: "alice"},
    redis.Z{Score: 8200, Member: "bob"},
    redis.Z{Score: 11000, Member: "charlie"},
)

// Get top 10 (highest score first)
leaders, _ := rdb.ZRevRangeWithScores(ctx, "game:leaderboard", 0, 9).Result()

// Get player rank (0-indexed)
rank, _ := rdb.ZRevRank(ctx, "game:leaderboard", "alice").Result()
// alice is rank 1 (charlie is 0)

// Rate limiting with sliding window
now := time.Now().UnixMilli()
window := int64(60000) // 60 seconds
rdb.ZAdd(ctx, "requests:user:42", redis.Z{Score: float64(now), Member: now})
rdb.ZRemRangeByScore(ctx, "requests:user:42", "0", strconv.FormatInt(now-window, 10))
count, _ := rdb.ZCard(ctx, "requests:user:42").Result()
if count > 100 { return ErrRateLimited }

// Scheduled tasks (score = execution time)
rdb.ZAdd(ctx, "scheduled_jobs", redis.Z{
    Score: float64(time.Now().Add(5*time.Minute).Unix()),
    Member: "job:send-email:42",
})
// Poll: get jobs due now
jobs, _ := rdb.ZRangeByScore(ctx, "scheduled_jobs", &redis.ZRangeBy{
    Min: "0",
    Max: strconv.FormatInt(time.Now().Unix(), 10),
    Count: 10,
}).Result()
```

Q75

What is Redis transaction (MULTI/EXEC)?

Difficulty:
Medium

```

// MULTI/EXEC: execute commands atomically (no interleaving)
// NOT isolation like SQL transactions
// All commands queued, executed atomically

// Basic transaction
pipe := rdb.TxPipeline()
pipe.Decr(ctx, "inventory:product:42")
pipe.Incr(ctx, "orders:count")
pipe.Set(ctx, "order:99:status", "confirmed", 0)
_, err := pipe.Exec(ctx)

// WATCH + MULTI/EXEC (optimistic locking)
// Retry if watched key changed (CAS pattern)
const maxRetries = 5
for i := 0; i < maxRetries; i++ {
    err := rdb.Watch(ctx, func(tx *redis.Tx) error {
        // Get current value
        n, err := tx.Get(ctx, "inventory").Int()
        if err != nil { return err }
        if n <= 0 { return errors.New("out of stock") }

        // Atomic decrement only if inventory unchanged
        _, err = tx.TxPipelined(ctx, func(pipe redis.Pipeliner) error {
            pipe.Decr(ctx, "inventory")
            return nil
        })
        return err
    }, "inventory")

    if err == nil { break }
    if err != redis.TxFailedErr { return err } // retry only on conflict
    time.Sleep(time.Duration(i) * 10 * time.Millisecond)
}

```

Q76

What is Redis connection pooling in Go?

Difficulty:
Medium

```
rdb := redis.NewClient(&redis.Options{
    Addr:      "localhost:6379",
    Password:  "secret",
    DB:        0,

    // Pool settings
    PoolSize:      10,          // max connections (default: 10 per CPU)
    MinIdleConns:  3,          // keep this many idle connections
    MaxIdleConns:  5,          // max idle
    PoolTimeout:   4 * time.Second, // wait for connection from pool
    ConnMaxIdleTime: 5 * time.Minute, // close idle connections after
    ConnMaxLifetime: 30 * time.Minute,

    // Timeouts
    DialTimeout:  5 * time.Second,
    ReadTimeout:  3 * time.Second,
    WriteTimeout: 3 * time.Second,
})

// Monitor pool stats
stats := rdb.PoolStats()
fmt.Printf("hits=%d misses=%d timeouts=%d total=%d idle=%d\n",
    stats.Hits, stats.Misses, stats.Timeouts,
    stats.TotalConns, stats.IdleConns)

// Health check
if err := rdb.Ping(ctx).Err(); err != nil {
    log.Fatal("redis ping failed:", err)
}
```

Q77

What is Redis keyspace notifications?

Difficulty:
Medium

```
// Keyspace notifications: get notified when keys expire, are set, deleted

// Enable in redis.conf or via CONFIG SET:
// notify-keyspace-events "Ex" # E=keyevent events, x=expired events
// notify-keyspace-events "KEA" # K=keyspace, E=keyevent, A=all events

rdb.ConfigSet(ctx, "notify-keyspace-events", "Ex")

// Subscribe to expiry events
pubsub := rdb.Subscribe(ctx, "__keyevent@0__:expired")
ch := pubsub.Channel()

for msg := range ch {
    expiredKey := msg.Payload
    fmt.Printf("key expired: %s\n", expiredKey)
    // Handle expiry: refresh cache, notify user, etc.
}

// Use cases:
// - Cache expiry hooks (warm up cache before item expires)
// - Session expiry notification (logout user)
// - Distributed lock expiry detection
// - TTL-based job scheduling

// Note: notifications are best-effort (may be lost)
// Don't use for critical workflows
```

Q78

What are Redis pipelines and batching?

Difficulty:
Medium

```
// Pipeline: send multiple commands in single round-trip
// No interleaving guarantee (vs MULTI/EXEC)

pipe := rdb.Pipeline()
for i := 0; i < 100; i++ {
    pipe.Set(ctx, fmt.Sprintf("key:%d", i), i, 0)
}
_, err := pipe.Exec(ctx)

// Pipelined (functional style)
cmds, err := rdb.Pipelined(ctx, func(pipe redis.Pipeliner) error {
    for i := 0; i < 100; i++ {
        pipe.Get(ctx, fmt.Sprintf("key:%d", i))
    }
    return nil
})
for _, cmd := range cmds {
    fmt.Println(cmd.(*redis.StringCmd).Val())
}

// Performance comparison:
// 100 individual GETs: 100 round trips × 0.1ms = 10ms
// 100 pipelined GETs: 1 round trip × 0.1ms + processing = ~1ms

// Cluster pipeline: routes to correct node automatically
pipe := rdb.(*redis.ClusterClient).Pipeline()
```

Q79

What is Redis memory management?

Difficulty: Hard

bash

```
# Memory info
redis-cli INFO memory
# used_memory: current memory
# used_memory_peak: peak memory
# mem_fragmentation_ratio: >1.5 = fragmented (defrag needed)

# Memory limit and eviction
CONFIG SET maxmemory 2gb
CONFIG SET maxmemory-policy allkeys-lru

# Eviction policies:
# noeviction:      error when memory full (default)
# allkeys-lru:     evict any key by LRU
# volatile-lru:    evict keys with TTL by LRU
# allkeys-random:  evict random key
# volatile-random: evict random key with TTL
# volatile-ttl:    evict key with shortest TTL
# allkeys-lfu:     evict by LFU (least frequently used, Redis 4+)
# volatile-lfu:    evict keys with TTL by LFU

# For cache: allkeys-lru or allkeys-lfu
# For session store: volatile-lru (keep keys without TTL)

# Check memory per key type
redis-cli --bigkeys # find largest keys
redis-cli MEMORY USAGE mykey # bytes used by specific key
```

go

```
// In Go: monitor memory
info, _ := rdb.Info(ctx, "memory").Result()
// Parse used_memory field
```

Q80

What is Redis Lua scripting?

Difficulty: Hard


```
// Lua scripts: atomic execution of multiple commands
// No interruption between commands (like transaction but with logic)

// Example: atomic increment with cap
luaScript := redis.NewScript(`
local current = tonumber(redis.call('GET', KEYS[1])) or 0
local max = tonumber(ARGV[1])
if current < max then
    return redis.call('INCR', KEYS[1])
end
return -1 -- at cap
`)

result, err := luaScript.Run(ctx, rdb, []string{"counter:api-calls"}, 100).Int()
if result == -1 {
    return ErrRateLimitExceeded
}

// Rate limiter using Lua (atomic check-and-increment)
rateLimitScript := `
local count = redis.call('INCR', KEYS[1])
if count == 1 then
    redis.call('EXPIRE', KEYS[1], ARGV[1])
end
return count
`

count, _ := rdb.Eval(ctx, rateLimitScript, []string{"rate:user:42"}, 60).Int()
if count > 100 { return ErrRateLimited }
```

Q81–Q100: Advanced Cache Patterns

Q81

What is a multi-level cache architecture?

L1: In-process cache (e.g., ristretto, bigcache)
 Access: ~1ns, size limited by service memory
 Scope: single pod only

L2: Redis (shared cache)
 Access: ~0.5ms, larger capacity
 Scope: all pods share same data

L3: Database
 Access: ~5ms (indexed), ~50ms (complex queries)
 Source of truth

Lookup order:

1. Check L1 (in-process)
2. Cache miss → check L2 (Redis)
3. Cache miss → query L3 (DB)
4. Populate L2, then L1

Invalidation:

On write → DELETE from L2 (all pods' L1 will expire naturally)
 Or: short TTL in L1 (accept some staleness)

```
func (c *MultiLevelCache) Get(ctx context.Context, key string) (string, error) {
    // L1 check
    if v, ok := c.l1.Get(key); ok { return v.(string), nil }

    // L2 check
    v, err := c.redis.Get(ctx, key).Result()
    if err == nil {
        c.l1.Set(key, v, 30*time.Second) // populate L1
        return v, nil
    }

    // DB
    v, err = c.db.Get(ctx, key)
    if err != nil { return "", err }
    c.redis.Set(ctx, key, v, 5*time.Minute) // populate L2
    c.l1.Set(key, v, 30*time.Second)       // populate L1
    return v, nil
}
```

Q82

What is consistent hashing for cache distribution?

Consistent hashing: distribute keys across nodes with minimal redistribution

Problem with simple $\text{hash}(\text{key}) \% N$:

Add/remove node $\rightarrow$ almost ALL keys remap $\rightarrow$ cache miss storm

Consistent hashing:

Virtual ring of 2^{32} slots

Nodes placed at multiple points (virtual nodes) on ring

Key $\rightarrow$ hash $\rightarrow$ clockwise to next node

Add node: only keys between new node and its predecessor remap

Remove node: only keys from removed node remap to next node

$\sim 1/N$ keys remap (vs $\sim N-1/N$ for simple hashing)

Virtual nodes: each physical node has 150+ virtual nodes

$\rightarrow$ more even distribution

$\rightarrow$ fewer hot spots

Libraries:

github.com/stathat/consistent (Go)

hashring pattern

Q83

What is Redis Geo for location-based queries?

```
// Redis GEO: store/query geographic coordinates
// Internally uses sorted set with geohash score

// Add locations
rdb.GeoAdd(ctx, "restaurants",
    &redis.GeoLocation{Name: "pizza-palace", Longitude: -122.4194, Latitude: 37.7749},
    &redis.GeoLocation{Name: "sushi-bar", Longitude: -122.4089, Latitude: 37.7851},
)

// Find nearby (within 5km)
results, _ := rdb.GeoRadius(ctx, "restaurants", -122.4194, 37.7749,
    &redis.GeoRadiusQuery{
        Radius:    5,
        Unit:      "km",
        WithDist:  true,
        WithCoord: true,
        Count:     10,
        Sort:      "ASC", // nearest first
    },
).Result()

for _, r := range results {
    fmt.Printf("%s: %.2f km away\n", r.Name, r.Dist)
}

// Distance between two points
dist, _ := rdb.GeoDist(ctx, "restaurants", "pizza-palace", "sushi-bar", "km").Result()
```

Q84

What is the Read-Aside vs Write-Through pattern?

Cache-Aside (Lazy Loading):

Read: check cache → miss → read DB → populate cache

Write: write to DB → invalidate/update cache

Pros: only cache what's needed, resilient to cache failure

Cons: cold start, thundering herd on first access

When: read-heavy, unpredictable access patterns

Write-Through:

Write: write to cache AND DB atomically (or DB first, then cache)

Read: always hits cache (freshest data)

Pros: cache always up-to-date, no stale reads

Cons: write latency (wait for both), cache bloat (write everything)

When: read-after-write consistency required, data always reused

Write-Behind (Write-Back):

Write: write to cache immediately, async flush to DB

Pros: ultra-fast writes

Cons: data loss if cache crashes before flush, complexity

When: high-write scenarios, eventual consistency acceptable

go

Q85

What is the cache warming strategy?

```
// Cache warming: pre-populate cache before going live
// Prevents cold start / thundering herd

// Strategy 1: Eager warming on startup
func warmCache(ctx context.Context, db *sql.DB, rdb *redis.Client) error {
    // Load top 1000 products (most accessed)
    products, err := db.QueryContext(ctx,
        "SELECT id,data FROM products ORDER BY view_count DESC LIMIT 1000")
    if err != nil { return err }

    pipe := rdb.Pipeline()
    for products.Next() {
        var id int64; var data []byte
        products.Scan(&id, &data)
        pipe.Set(ctx, fmt.Sprintf("product:%d", id), data, time.Hour)
    }
    _, err = pipe.Exec(ctx)
    return err
}

// Strategy 2: Background warming (don't block startup)
go func() {
    time.Sleep(5 * time.Second) // let service start first
    warmCache(ctx, db, rdb)
}()

// Strategy 3: Predictive warming (next likely-needed keys)
// After serving product:42, pre-warm related products
```

Q86

What is Redis TTL management best practices?

```
// TTL strategies:
// 1. Absolute TTL: key expires at fixed time
rdb.Set(ctx, "session:abc", data, 30*time.Minute)

// 2. Sliding TTL: reset on each access
func getWithSlidingTTL(ctx context.Context, key string) (string, error) {
    pipe := rdb.Pipeline()
    get := pipe.Get(ctx, key)
    pipe.Expire(ctx, key, 30*time.Minute) // reset TTL on access
    pipe.Exec(ctx)
    return get.Val(), get.Err()
}

// 3. TTL jitter: prevent thundering herd (all keys expiring together)
func setWithJitter(ctx context.Context, key string, val interface{}, base time.Duration) error {
    jitter := time.Duration(rand.Intn(int(base / 10))) // ±10%
    return rdb.Set(ctx, key, val, base+jitter).Err()
}

// Check remaining TTL
ttl, _ := rdb.TTL(ctx, "session:abc").Result()
if ttl < 5*time.Minute {
    // Proactively refresh before expiry
    refreshSession(ctx, key)
}
```

Q87

What is the session store pattern with Redis?

```
// Redis as centralized session store

type Session struct {
    UserID      int64
    ExpiresAt   time.Time
    Data        map[string]interface{}
}

func createSession(ctx context.Context, rdb *redis.Client, userID int64) (string, error) {
    sessionID := generateSecureToken()
    session := Session{
        UserID:      userID,
        ExpiresAt:    time.Now().Add(24 * time.Hour),
        Data:        make(map[string]interface{}),
    }

    data, _ := json.Marshal(session)
    err := rdb.Set(ctx, "session:"+sessionID, data, 24*time.Hour).Err()
    return sessionID, err
}

func getSession(ctx context.Context, rdb *redis.Client, sessionID string) (*Session, error) {
    data, err := rdb.Get(ctx, "session:"+sessionID).Bytes()
    if errors.Is(err, redis.Nil) { return nil, ErrSessionNotFound }
    if err != nil { return nil, err }

    var session Session
    json.Unmarshal(data, &session)
    return &session, nil
}

// Sticky sessions vs centralized:
// Sticky: user always hits same server (simpler but hard to scale)
// Centralized Redis: any server handles any user (horizontal scale)
```

Q88

What is distributed rate limiting with Redis?

```

go
// Sliding window rate limiter (most accurate)
func isRateLimited(ctx context.Context, rdb *redis.Client, key string, limit int, window time.Duration) (bool, error) {
    now := time.Now().UnixMilli()
    windowMs := window.Milliseconds()

    pipe := rdb.TxPipeline()
    pipe.ZRemRangeByScore(ctx, key, "0", strconv.FormatInt(now-windowMs, 10)) // remove old
    pipe.ZAdd(ctx, key, redis.Z{Score: float64(now), Member: now}) // add current
    countCmd := pipe.ZCard(ctx, key) // count in window
    pipe.Expire(ctx, key, window) // set TTL
    _, err := pipe.Exec(ctx)
    if err != nil { return false, err }

    count, _ := countCmd.Result()
    return count > int64(limit), nil
}

// Token bucket (bursting allowed)
// Fixed window (simpler, less accurate)
// Leaky bucket (smooth output rate)

// Choose based on requirements:
// API rate limiting: sliding window (most fair)
// Burst tolerance: token bucket
// Simple protection: fixed window

```

Q89

What is Redis ACL (Access Control Lists)?

```

bash
# Redis 6.0+ ACL: fine-grained access control

# Create user
ACL SETUSER alice on >password ~cache:* +get +set +del
# alice: enabled, password "password", can only access keys matching "cache:*"
# can only use GET, SET, DEL commands

ACL SETUSER readonly on >pass ~* +@read # read-only on all keys
ACL SETUSER admin on >pass ~* +@all # full access

# In Go
rdb := redis.NewClient(&redis.Options{
    Addr: "localhost:6379",
    Username: "alice",
    Password: "password",
})

# List users
ACL LIST
ACL WHOAMI
ACL LOG # log auth failures

```


Q90

What is Redis keyspace design best practices?

Key naming conventions:

```
<object>:<id>                user:42
<object>:<id>:<field>          user:42:profile
<namespace>:<object>:<id>     myapp:prod:user:42
```

Avoid:

- Very long keys (memory overhead)
- JSON as key (use hash tags for clustering)
- Spaces in keys (use _ or :)

Key size:

- Keys: keep < 1KB (ideally < 128 bytes)
- Values: depends on use case
- Large values: consider compression (msgpack vs JSON)

Naming for different data types:

String:	user:42	(serialized JSON)
Hash:	user:42:fields	(individual fields)
Set:	user:42:friends	(friend IDs)
List:	user:42:feed	(recent activity)
ZSet:	leaderboard	(score-based)
Stream:	events:orders	(event log)

Key expiry:

- Always set TTL for cache keys
- No TTL for persistent data (sessions with explicit logout)
- Monitor keys without TTL: `redis-cli --scan --pattern "cache:*`

Q91

What are Redis data structure memory optimization?

Encoding optimization:

Small hashes use ziplist (< hash-max-ziplist-entries=128):
hash-max-ziplist-entries 128
hash-max-ziplist-value 64
→ Saves 10x memory vs hash table

Small lists use ziplist:

list-max-ziplist-size -2 (-2 = 8KB max)

Small sets of integers use intset:

set-max-intset-entries 512

Small sorted sets use ziplist:

zset-max-ziplist-entries 128
zset-max-ziplist-value 64

Object encoding check:

DEBUG OBJECT mykey → encoding: ziplist | hashtable | intset | etc.

OBJECT ENCODING mykey

Compression:

Compress values with gzip/snappy before storing
Trade CPU for memory
Worth it for values > 1KB

redis-cli --memkeys: analyze memory usage by key pattern

Q92

What is cache consistency in distributed systems?

Consistency problems:

Write races: two writers update cache and DB in different orders
Stale reads: cache has old value after DB update

Solutions:

1. Cache-aside with TTL (accept eventual consistency):
Read from cache **if** fresh, **else** DB
Stale window = TTL
Simple, works **for** most cases
2. Write-invalidate (most reliable):
Write to DB → DELETE cache key
Next read fetches fresh from DB
Problem: delete-before-write race (two processes)
3. Write-invalidate with version:
DB row has version column
Cache stores: {"**version**": 5, "**data**": {...}}
On cache hit: compare version with DB (occasional check)
4. CDC-based invalidation:
Debezium reads DB WAL → invalidates Redis keys
Eventually consistent, no application-level sync
5. Read-your-writes consistency:
After write: route reads to primary DB (bypass cache)
After TTL or delay: **switch** back to cache reads
Use sticky routing or version token

Q93

What is Redis keyspace memory analysis?

```
# Memory analysis tools

# 1. redis-cli --bigkeys (find largest keys)
redis-cli --bigkeys
# Samples: 100 keys, finds largest per type

# 2. redis-cli --memkeys (Go 1.x, newer redis-cli)
redis-cli -u redis://localhost:6379 --memkeys

# 3. MEMORY USAGE (individual key)
redis-cli MEMORY USAGE user:42 # bytes used by key + value

# 4. SCAN + MEMORY USAGE (script all keys in pattern)
redis-cli --scan --pattern "session:*" | xargs -L 1 redis-cli MEMORY USAGE

# 5. RDB analysis (rdbtools)
rdb --command memory dump.rdb | sort -t, -k4 -rn | head -20

# Common memory hogs:
# Large string values (JSON blobs)
# Unbounded lists/sets
# Keys without TTL accumulating
# Hash fields explosion (1M fields in one hash)
```

Q94

What is Redis eviction and OOM handling?

```
// Handle Redis OOM in application

func setCache(ctx context.Context, key string, val interface{}, ttl time.Duration) error {
    data, _ := json.Marshal(val)
    err := rdb.Set(ctx, key, data, ttl).Err()
    if err != nil {
        if strings.Contains(err.Error(), "OOM") {
            // Redis is out of memory
            // Log but don't fail the request (cache is optional)
            log.Printf("cache OOM, skipping cache write for key %s", key)
            return nil // graceful degradation
        }
        return err
    }
    return nil
}

// Monitor eviction rate
// redis-cli INFO stats | grep evicted_keys
// Alert if evicted_keys rate > 0 (means maxmemory hit)

// Set appropriate maxmemory:
// total_memory × 0.75 (leave headroom for fragmentation)
// Example: 8GB Redis → maxmemory 6gb

// Monitor with Prometheus:
// redis_evicted_keys_total (counter, alert on rate > 0)
// redis_memory_used_bytes / redis_memory_max_bytes (usage ratio, alert >80%)
```

Q95

What is Redis replication lag?

```
# Check replication status
redis-cli INFO replication
# role: master
# connected_slaves: 2
# slave0: ip=...,port=...,state=online,offset=123456,lag=0
# slave1: ip=...,port=...,state=online,offset=123450,lag=2

# lag=0: fully caught up
# lag=2: 2 seconds behind (normal under load)

# master_repl_offset: current master offset
# slave0.offset: replica's confirmed offset
# Lag bytes = master_repl_offset - slave0.offset

# Monitoring:
# redis_connected_slaves (alert if drops below expected)
# redis_replication_backlog_size
# slave lag (from INFO replication parsing)

# Replication buffer
config set repl-backlog-size 1mb # buffer for disconnected replicas

# Partial resync: replica reconnects, catches up from backlog
# Full resync: if backlog too small, full RDB copy needed (expensive)
```

Q96

What is in-memory cache libraries in Go?

```
// Options:

// 1. ristretto (Cloudflare): most production-grade
import "github.com/dgraph-io/ristretto"

cache, _ := ristretto.NewCache(&ristretto.Config{
    NumCounters: 1e7,      // track 10M items for admission
    MaxCost:     1 << 30, // 1GB max
    BufferItems:  64,      // performance tuning
})
cache.Set("key", value, cost) // cost = estimated memory
val, found := cache.Get("key")

// 2. bigcache (Allegro): for millions of small items
import "github.com/allegro/bigcache/v3"

cache, _ := bigcache.New(context.Background(), bigcache.DefaultConfig(10*time.Minute))
cache.Set("key", []byte(value))
entry, _ := cache.Get("key")

// 3. groupcache: distributed cache without Redis
// Perfect for read-heavy, shared computation

// 4. freecache: zero GC overhead
import "github.com/coocood/freecache"
cache := freecache.NewCache(100 * 1024 * 1024) // 100MB
cache.Set([]byte("key"), []byte("val"), 300)
```

Q97

What is the cache stampede (thundering herd) and prevention?

```
// Cache stampede: many requests hit DB simultaneously when cache expires

// Solution 1: singleflight (coalesce identical requests)
import "golang.org/x/sync/singleflight"

var sfGroup singleflight.Group

func getUser(ctx context.Context, id int64) (*User, error) {
    key := fmt.Sprintf("user:%d", id)

    result, err, _ := sfGroup.Do(key, func() (interface{}, error) {
        // Only ONE goroutine runs this for the same key
        // Others wait and share the result
        user, err := db.GetUser(ctx, id)
        if err != nil { return nil, err }
        redis.Set(ctx, key, user, time.Hour)
        return user, nil
    })
    if err != nil { return nil, err }
    return result.(*User), nil
}

// Solution 2: mutex per key (Redis SETNX lock)
func getWithLock(ctx context.Context, key string) (string, error) {
    val, err := rdb.Get(ctx, key).Result()
    if err == nil { return val, nil } // cache hit

    // Try to acquire lock
    locked, _ := rdb.SetNX(ctx, key+":lock", "1", 10*time.Second).Result()
    if !locked {
        time.Sleep(50 * time.Millisecond)
        return rdb.Get(ctx, key).Result() // wait and retry
    }
    defer rdb.Del(ctx, key+":lock")

    // We have the lock: compute value
    val = fetchFromDB()
    rdb.Set(ctx, key, val, time.Hour)
    return val, nil
}

// Solution 3: probabilistic early expiration (PER)
// Refresh cache slightly before expiry with probability
```

Q98

What is Redis TLS configuration?


```
import "crypto/tls"

rdb := redis.NewClient(&redis.Options{
    Addr: "redis.prod.example.com:6380",
    TLSConfig: &tls.Config{
        MinVersion: tls.VersionTLS12,
        // For custom CA:
        RootCAs: loadCACert("redis-ca.pem"),
        // For mTLS (client cert):
        Certificates: []tls.Certificate{loadClientCert()},
    },
})

// Redis Cluster with TLS
rdb := redis.NewClusterClient(&redis.ClusterOptions{
    Addrs: []string{"7000", "7001", "7002"},
    TLSConfig: &tls.Config{MinVersion: tls.VersionTLS12},
})

// redis.conf:
// tls-port 6380
// tls-cert-file /etc/ssl/redis.crt
// tls-key-file /etc/ssl/redis.key
// tls-ca-cert-file /etc/ssl/ca.crt
// tls-auth-clients no # optional client certs
```

Q99

What are Redis performance benchmarks?

```
# Redis benchmark tool
redis-benchmark -q -n 100000
# -n: number of requests
# -c: concurrent connections (default 50)
# -q: quiet (only summary)

redis-benchmark -q -n 100000 -c 100 -t get,set,incr,lpush

# Typical results (local, single thread):
# GET: ~110,000 ops/sec
# SET: ~110,000 ops/sec
# INCR: ~110,000 ops/sec
# LPUSH: ~100,000 ops/sec

# With pipeline (batch)
redis-benchmark -n 100000 -P 16 # pipeline 16 cmds
# GET: ~1,000,000 ops/sec (10x improvement)

# Latency percentiles
redis-benchmark --latency-history -t get -i 1
redis-cli --latency-history # live latency monitoring

# Redis is single-threaded for commands:
# I/O multiplexing handles many connections
# Commands serialize on main thread
# Cluster: scale horizontally per shard
```

Q100

What is the cache invalidation strategy for microservices?

Cache invalidation is hard ("one of two hard problems in CS")

Strategies for microservices:

1. TTL-based (simplest):

Set short TTL, accept staleness window

No invalidation needed – just wait for expiry

Good for: product catalog, configs, non-critical data

2. Event-driven invalidation:

Service A updates DB → publishes "user:updated" event

Service B (cache owner) subscribes → deletes/refreshes cache

Requires: message broker (Kafka/RMQ)

Good for: cross-service cache dependencies

3. Write-through invalidation:

Update DB → immediately invalidate/update cache

In same transaction or 2-phase with retry

Good for: single-service, same-DB ownership

4. CDC-based (most reliable):

Debezium reads DB WAL → publishes changes → cache invalidator subscribes

Decoupled from application code

Near-real-time

Good for: large-scale, multi-consumer invalidation

5. Version/ETag based:

Cache stores version alongside data

Check version on read, invalidate if stale

Good for: API caching, HTTP caching

Rule: cache owner is responsible for invalidation

Never let service B invalidate service A's cache

Extended Questions (Q102–Q150)

Q102

What is ristretto cache in Go?

Difficulty:
Medium

```

import "github.com/dgraph-io/ristretto"

// Ristretto: high-performance in-process cache for Go
// Features: admission policy (TinyLFU), per-item cost, metrics

cache, err := ristretto.NewCache(&ristretto.Config{
    NumCounters: 1e7,          // 10M counters for admission tracking
    MaxCost:     1 << 30,      // 1GB max total cost
    BufferItems:  64,          // async operations per goroutine
    Metrics:     true,         // enable metrics
})
if err != nil { panic(err) }
defer cache.Close()

// Set with cost (estimated memory bytes)
cost := int64(len(key) + len(value))
cache.Set(key, value, cost)
cache.SetWithTTL(key, value, cost, time.Hour)

// Get
val, found := cache.Get(key)
if found {
    use(val.(string))
}

// Wait for async operations to complete (in tests)
cache.Wait()

// Metrics
m := cache.Metrics
fmt.Printf("hits=%d misses=%d ratio=%.2f\n",
    m.Hits(), m.Misses(), m.Ratio())

// Eviction callback
config.OnEvict = func(item *ristretto.Item) {
    log.Printf("evicted: key=%v", item.Key)
}

```

Q103

What is bigcache in Go?

Difficulty:
Medium

```

import "github.com/allegro/bigcache/v3"

// BigCache: fast, GC-friendly in-process cache
// Key feature: stores values as []byte → no GC pressure from interface{}
// Suitable for: millions of small items

config := bigcache.DefaultConfig(10 * time.Minute)
config.MaxEntriesInWindow = 1000 * 10 * 60
config.MaxEntrySize = 500           // bytes
config.HardMaxCacheSize = 256       // MB
config.OnRemove = func(key string, entry []byte) {
    log.Printf("removed: %s", key)
}

cache, err := bigcache.New(context.Background(), config)
if err != nil { panic(err) }
defer cache.Close()

// Store bytes
cache.Set("user:42", jsonBytes)

// Get bytes
entry, err := cache.Get("user:42")
if errors.Is(err, bigcache.ErrEntryNotFound) {
    // cache miss
}

// Delete
cache.Delete("user:42")

// Stats
stats := cache.Stats()
fmt.Printf("hits=%d misses=%d\n", stats.Hits, stats.Misses)

// Limitations:
// No per-item TTL (global TTL only)
// No cost-based eviction
// LRU-ish eviction (hash segments)

```

Q104

What is cache-aside pattern in Go?

Difficulty:
Medium

```

type UserCache struct {
    redis *redis.Client
    db     *pgxpool.Pool
    ttl    time.Duration
}

func (c *UserCache) GetUser(ctx context.Context, id int64) (*User, error) {
    key := fmt.Sprintf("user:%d", id)

    // 1. Try cache
    data, err := c.redis.Get(ctx, key).Bytes()
    if err == nil {
        var user User
        if err := json.Unmarshal(data, &user); err == nil {
            return &user, nil
        }
    }

    // 2. Cache miss → DB
    var user User
    err = c.db.QueryRow(ctx, "SELECT id,name,email FROM users WHERE id=$1", id).
        Scan(&user.ID, &user.Name, &user.Email)
    if errors.Is(err, pgx.ErrNoRows) {
        // Cache negative result to prevent DB hammering
        c.redis.Set(ctx, key+":notfound", "1", time.Minute)
        return nil, ErrNotFound
    }
    if err != nil { return nil, err }

    // 3. Populate cache
    data, _ = json.Marshal(user)
    c.redis.Set(ctx, key, data, c.ttl)

    return &user, nil
}

func (c *UserCache) UpdateUser(ctx context.Context, user *User) error {
    _, err := c.db.Exec(ctx, "UPDATE users SET name=$1,email=$2 WHERE id=$3",
        user.Name, user.Email, user.ID)
    if err != nil { return err }

    // Invalidate cache
    c.redis.Del(ctx, fmt.Sprintf("user:%d", user.ID))
    return nil
}

```

Q105

What is negative caching?

Difficulty:
Medium

```
// Negative caching: cache "not found" results to prevent DB hammering
// Problem: DoS via non-existent keys → every request hits DB

func (c *Cache) GetProduct(ctx context.Context, id int64) (*Product, error) {
    key := fmt.Sprintf("product:%d", id)
    negKey := fmt.Sprintf("product:%d:notfound", id)

    // Check negative cache first
    if c.redis.Exists(ctx, negKey).Val() > 0 {
        return nil, ErrNotFound // fast return, no DB hit
    }

    // Check positive cache
    data, err := c.redis.Get(ctx, key).Bytes()
    if err == nil {
        var p Product
        json.Unmarshal(data, &p)
        return &p, nil
    }

    // DB lookup
    product, err := c.db.GetProduct(ctx, id)
    if errors.Is(err, pgx.ErrNoRows) {
        // Cache the "not found" with shorter TTL
        c.redis.Set(ctx, negKey, "1", 30*time.Second) // short TTL
        return nil, ErrNotFound
    }
    if err != nil { return nil, err }

    // Cache positive result
    data, _ = json.Marshal(product)
    c.redis.Set(ctx, key, data, time.Hour)
    return product, nil
}
```

Q106

What is cache penetration vs avalanche vs breakdown?

Difficulty: Hard

Cache Penetration:

Definition: query `for` non-existent key → always hits DB

Cause: malicious requests with fake IDs, deleted data

Solution:

1. Negative caching: cache `"not found"` with short TTL
2. Bloom filter: fast check `if` key exists before hitting cache/DB
3. Input validation: reject obviously invalid IDs

Cache Avalanche:

Definition: many keys expire at same time → traffic floods DB

Cause: mass deployment (all keys set with same TTL), power outage

Solution:

1. TTL jitter: add random offset (`TTL + rand(0, 10%*TTL)`)
2. Gradual expiry: different TTLs `for` different key groups
3. Circuit breaker: throttle DB when overloaded
4. Multi-level cache: L1 in-process + L2 Redis

Cache Breakdown (hot key expiry):

Definition: one hot key expires → thundering herd on that key

Cause: viral post, trending product, live event

Solution:

1. Mutex/singleflight: only one goroutine rebuilds
2. Never expire hot keys (or very long TTL)
3. Pre-warm before expiry (probabilistic early refresh)
4. Background async refresh (serve stale while refreshing)

Q107

What is Redis cluster hash slot migration?

Difficulty: Hard


```
# Resharding: move hash slots between nodes (for rebalancing)
redis-cli --cluster reshard 127.0.0.1:7000

# Interactive: how many slots? from which node? to which node?
# Or automated:
redis-cli --cluster reshard 127.0.0.1:7000 \
  --cluster-from source-node-id \
  --cluster-to target-node-id \
  --cluster-slots 1000 \
  --cluster-yes

# Add new node (scale out)
redis-cli --cluster add-node 127.0.0.1:7006 127.0.0.1:7000
# Then reshard some slots to new node

# Remove node (scale in)
# First: move all its slots to other nodes
redis-cli --cluster reshard ... # move slots away
# Then remove:
redis-cli --cluster del-node 127.0.0.1:7000 node-id

# During migration:
# MOVED error: key moved to different node (client redirects)
# ASK error: key being migrated (temporary redirect)
# go-redis handles both automatically (smart client)

# Check cluster balance
redis-cli --cluster check 127.0.0.1:7000
redis-cli --cluster info 127.0.0.1:7000
```

Q108

What is Redis RESP protocol?

Difficulty: Hard

RESP (Redis Serialization Protocol): simple, binary-safe protocol

Types:

```
+ Simple string: +OK\r\n
- Error:         -ERR unknown command\r\n
: Integer:       :1000\r\n
$ Bulk string:   $6\r\nfoobar\r\n
                 $-1\r\n (null bulk string)
* Array:         *3\r\n:1\r\n:2\r\n:3\r\n ([1,2,3])
```

Example: SET key value

```
Client sends: *3\r\n$3\r\nSET\r\n$3\r\nkey\r\n$5\r\nvalue\r\n
Server responds: +OK\r\n
```

Example: GET key

```
Client: *2\r\n$3\r\nGET\r\n$3\r\nkey\r\n
Server: $5\r\nvalue\r\n (or $-1\r\n for nil)
```

RESP3 (Redis 6.0+):

```
New types: Map, Set, Double, Boolean, BigNumber, VerbatimString
Push messages (for pub/sub, keyspace notifications)
go-redis v9: supports RESP3 automatically
```

Pipelining efficiency:

```
Send N commands without waiting for responses
Server processes in order, sends all responses
1 RTT instead of N RTTs
```

Q109

What is Redis Object Encoding internals?

Difficulty: Hard

```
# Redis uses different internal encodings based on value characteristics
# Check encoding:
OBJECT_ENCODING mykey

# String encodings:
# int:         integer <= 2^63-1 (e.g., INCR counter)
# embstr:      string <= 44 bytes (single allocation, immutable)
# raw:         string > 44 bytes (two allocations, mutable)

# Hash encodings:
# ziplist → listpack: <= hash-max-listpack-entries (128) and value <= 64 bytes
# hashtable: when exceeds thresholds (10-15x larger!)

# List encodings:
# listpack: <= list-max-listpack-size
# quicklist: when exceeds (doubly-linked list of listpacks)

# Set encodings:
# listpack: <= set-max-listpack-entries (128) and values <= 64 bytes
# intset: all integers <= set-max-intset-entries (512)
# hashtable: otherwise

# Sorted Set encodings:
# listpack: <= zset-max-listpack-entries (128)
# skiplist + hashtable: when exceeds

# Performance impact:
# ziplist/listpack: compact, cache-friendly, O(n) operations
# hashtable: O(1) lookup, more memory
# Threshold tuning: larger thresholds → more memory savings but slower operations
```

Q110

What is Redis pub/sub vs Streams comparison?

Difficulty:
Medium

Redis Pub/Sub:

Pattern: fire-and-forget publish/subscribe
Persistence: NONE (if no subscribers → message lost)
Consumer groups: NO (all subscribers get all messages)
Replay: IMPOSSIBLE (no history)
Delivery: at-most-once
Use: real-time notifications, live chat, metrics broadcasting

Redis Streams:

Pattern: persistent, ordered log
Persistence: YES (stored until XDEL or XTRIM)
Consumer groups: YES (like Kafka groups, each group independent)
Replay: YES (read from any position)
Delivery: at-least-once (with ACK)
Pending entries: track unacked messages per consumer
Use: reliable event log, task queues, audit trail

When to use Pub/Sub:

Live dashboard (stale data ok if missed)
Cache invalidation notifications (TTL handles stale anyway)
Chat messages (missed = user reconnects and loads from DB)

When to use Streams:

Task queue (can't lose tasks)
Order events (need replay for new consumers)
Audit logging
Replacing Kafka for small-scale event streaming

Code comparison:

Pub/Sub: SUBSCRIBE channel, PUBLISH channel message
Streams: XADD stream * field1 val1, XREADGROUP GROUP g consumer STREAMS stream >

Q111

What is cache serialization formats?

Difficulty:
Medium

```
// JSON: human readable, universal, slower
data, _ := json.Marshal(user)           // ~200 ns
json.Unmarshal(data, &user)             // ~300 ns
// Size: ~100 bytes for simple struct

// MessagePack: binary JSON, 20-30% smaller, 2-3x faster
import "github.com/vmihailov/msgpack"
data, _ := msgpack.Marshal(user)         // ~80 ns
msgpack.Unmarshal(data, &user)          // ~120 ns
// Size: ~70 bytes

// Protocol Buffers: fastest, smallest, requires schema
import "google.golang.org/protobuf/proto"
data, _ := proto.Marshal(userProto)      // ~50 ns
proto.Unmarshal(data, userProto)         // ~80 ns
// Size: ~50 bytes

// GOB: Go-specific, no schema needed
var buf bytes.Buffer
gob.NewEncoder(&buf).Encode(user)
gob.NewDecoder(&buf).Decode(&user)
// Comparable to JSON but Go-only

// Recommendation for Redis cache:
// High throughput, simple types: MessagePack
// Cross-language compatibility: JSON or Protobuf
// Go-only microservice: MessagePack or GOB
// Complex schemas, many services: Protobuf
```

Q112

What is Redis geo commands for location services?

Difficulty:
Medium

```
// GEO: location-based features (restaurant finder, delivery tracking)

// Add locations (lon, lat order!)
rdb.GeoAdd(ctx, "drivers", &redis.GeoLocation{
    Name:      "driver:42",
    Longitude: 77.5946, // Delhi longitude
    Latitude:  12.9716, // Bangalore latitude
})

// Update driver location
rdb.GeoAdd(ctx, "drivers", &redis.GeoLocation{
    Name:      "driver:42",
    Longitude: 77.5946 + 0.01,
    Latitude:  12.9716 + 0.01,
})

// Find drivers within 5km of customer
results, _ := rdb.GeoSearch(ctx, "drivers", &redis.GeoSearchQuery{
    Member:      "customer:99", // search around this member
    // OR: Longitude + Latitude for a point
    RadiusMiles: 5,
    Sort:        "ASC",
    Count:       10,
}).Result()

// GeoSearchStore: store results in another sorted set
rdb.GeoSearchStore(ctx, "drivers", "nearby_drivers:99", &redis.GeoSearchStoreQuery{
    GeoSearchQuery: redis.GeoSearchQuery{Longitude: 77.59, Latitude: 12.97, RadiusKM: 5, Sort: "ASC"},
    StoreDist:      true,
})

// Distance
dist, _ := rdb.GeoDist(ctx, "drivers", "driver:42", "driver:43", "km").Result()
fmt.Printf("distance: %.2f km\n", dist)

// Get coordinates
pos, _ := rdb.GeoPos(ctx, "drivers", "driver:42").Result()
fmt.Printf("lat=%.4f lon=%.4f\n", pos[0].Latitude, pos[0].Longitude)
```

Q113

What is Redis WAIT command for replication sync?

Difficulty: Hard

```
// WAIT: block until N replicas acknowledged writes
// Use: read-after-write consistency in Redis with replicas

// After critical write: ensure N replicas have it
numReplicas, err := rdb.Wait(ctx, 1, 1000).Result()
// Wait for 1 replica to ack, timeout 1000ms
// Returns: number of replicas that acknowledged (may be 0 if timeout)

if numReplicas < 1 {
    // Replica didn't ack in time – decide: proceed anyway or error?
    log.Printf("replica sync timeout, proceeding with %d replicas", numReplicas)
}

// Use case: payment service
func processPayment(ctx context.Context, rdb *redis.Client, payment Payment) error {
    // Write payment to Redis
    rdb.HSet(ctx, "payment:"+payment.ID, payment.ToMap())

    // Ensure at least 1 replica has it before returning to client
    n, _ := rdb.Wait(ctx, 1, 500).Result()
    if n == 0 {
        log.Warn("payment not yet replicated, continuing")
    }

    return nil
}

// WAIT overhead: adds latency equal to replication lag
// Typical: 1-5ms in same datacenter
// Trade-off: stronger durability vs higher latency
```

Q114

What is Redis memory optimization with encoding thresholds?

Difficulty: Hard

```
# Encoding thresholds control memory vs performance trade-off

# Hash: use listpack (compact) for small hashes
hash-max-listpack-entries 128 # (default, up to 512 saves significant memory)
hash-max-listpack-value 64    # max value size in listpack

# Experiment: storing user sessions as hashes
# user:42 = {session_id, token, user_id, created_at, ...}
# With listpack: ~200 bytes per user session
# With hashtable: ~800 bytes per user session
# Savings: 75% memory at 128 entries threshold

# Sorted set: use listpack for leaderboards with few entries
zset-max-listpack-entries 128
zset-max-listpack-value 64

# Set: intset for integer sets
set-max-intset-entries 512 # pure integer sets up to 512 elements

# List: quicklist node size
list-max-listpack-size -2 # -2 = 8KB per node

# Test memory usage
redis-cli --bigkeys # find keys using most memory
redis-cli MEMORY USAGE mykey # bytes for specific key
redis-cli OBJECT ENCODING mykey # current encoding

# Memory fragmentation
INFO memory | grep mem_fragmentation_ratio
# > 1.5: high fragmentation → MEMORY PURGE or restart
redis-cli MEMORY PURGE # release fragmented memory (Redis 4.0+)
```

Q115

What is multi-region cache architecture?

Difficulty: Hard

Problem: single Redis region → high latency for distant users

Architecture options:

1. Read replicas in each region:

Master (us-east-1) → Replica (eu-west-1) → Replica (ap-southeast-1)

Writes: always go to master (cross-region latency)

Reads: from local replica (low latency)

Consistency: eventual (replica lag 5-50ms)

2. Region-local caches:

Each region has independent Redis

Cache miss: read from local DB replica

Invalidation: cross-region event (Kafka, SNS)

Consistency: eventual (event propagation lag)

3. CRDTs (Conflict-free Replicated Data Types):

Redis Enterprise: active-active geo-distribution

Each region accepts writes, CRDTs merge conflicts

Suitable: counters, sets, sorted sets

Cache warmup on region startup:

New region: cold cache → thundering herd

Solution: warm from S3 snapshot or pre-fetch top N keys

Cross-region latency costs:

us-east → eu-west: ~80ms

us-east → ap-southeast: ~200ms

Same region: <1ms

→ Never do cross-region cache reads in hot path!

Q116

What are cache metrics and alerting?

Difficulty:
Medium

```
// Key cache metrics to track:

// 1. Hit rate (most important)
hitRate := float64(cacheHits) / float64(cacheHits+cacheMisses)
// Alert: hit rate < 80% (depends on use case)

// 2. Latency percentiles
// p50 < 1ms, p99 < 5ms for Redis

// 3. Memory usage
// Alert: > 80% of maxmemory

// 4. Eviction rate
// redis-cli INFO stats | grep evicted_keys
// Alert: evicted_keys > 0 (means maxmemory reached)

// 5. Connection pool utilization
// Alert: > 80% of pool size used consistently

// Prometheus metrics (go-redis built-in)
rdb := redis.NewClient(&redis.Options{...})
collector := redisprometheus.NewCollector("namespace", "subsystem", rdb)
prometheus.MustRegister(collector)

// Grafana dashboard metrics:
// redis_connected_clients
// redis_memory_used_bytes / redis_memory_max_bytes
// redis_keyspace_hits_total / (redis_keyspace_hits_total + redis_keyspace_misses_total)
// redis_evicted_keys_total
// redis_commands_duration_seconds_bucket (latency histogram)

// Application-level metrics
var (
    cacheHits = promauto.NewCounter(prometheus.CounterOpts{Name: "cache_hits_total"})
    cacheMisses = promauto.NewCounter(prometheus.CounterOpts{Name: "cache_misses_total"})
    cacheLatency = promauto.NewHistogram(prometheus.HistogramOpts{
        Name:    "cache_operation_duration_seconds",
        Buckets: []float64{.0005, .001, .005, .01, .05},
    })
)
```

Q117

What is cache testing strategies?

Difficulty:
Medium

```
// 1. Unit test with mock cache
type MockCache struct {
    data map[string]interface{}
    mu    sync.Mutex
}

func (m *MockCache) Get(key string) (interface{}, bool) {
    m.mu.Lock(); defer m.mu.Unlock()
    v, ok := m.data[key]
    return v, ok
}

func (m *MockCache) Set(key string, val interface{}, ttl time.Duration) {
    m.mu.Lock(); defer m.mu.Unlock()
    m.data[key] = val
}

// 2. Integration test with miniredis
import "github.com/alicebob/miniredis/v2"

func TestWithRedis(t *testing.T) {
    mr := miniredis.RunT(t) // in-memory Redis server for tests
    rdb := redis.NewClient(&redis.Options{Addr: mr.Addr()})

    svc := NewService(rdb)

    // Test cache hit
    mr.Set("user:42", `{"id":42,"name":"Alice"}`)
    user, err := svc.GetUser(ctx, 42)
    assert.NoError(t, err)
    assert.Equal(t, "Alice", user.Name)

    // Test TTL
    mr.FastForward(time.Hour + time.Second)
    assert.False(t, mr.Exists("user:42"))

    // Test cache miss → DB fallback
    user, err = svc.GetUser(ctx, 99)
    assert.Equal(t, ErrNotFound, err)
}

```

Q118–Q150: Final Cache Questions

Q118

What is distributed lock with Redlock?

```
// Redlock: distributed lock using N Redis nodes (usually 5)
// Majority of nodes must grant lock
import "github.com/go-redis/redis/v4"

pool := goredis.NewPool(rdb)
rs := redsync.New(pool)

mutex := rs.NewMutex("critical-section",
    redsync.WithExpiry(15*time.Second),
    redsync.WithRetryDelay(100*time.Millisecond),
    redsync.WithTries(32),
)

if err := mutex.Lock(); err != nil {
    return ErrCouldNotLock
}
defer mutex.Unlock()
// Critical section here
```

Q119

What is Redis backup and disaster recovery?

```
# RDB backup (point-in-time snapshot)
redis-cli BGSAVE          # async background save
redis-cli LASTSAVE        # timestamp of last save
# File: /var/lib/redis/dump.rdb

# AOF backup (append-only log)
redis-cli BGREWRITEAOF    # compact AOF file

# Automated backup with cron
0 * * * * redis-cli BGSAVE && cp /var/redis/dump.rdb /backup/redis-$(date +%Y%m%d%H).rdb

# Cloud: Redis ElastiCache automated backups
# Backup window: set in cluster config
# Retention: 1-35 days
# Restore: creates new cluster from backup

# DR strategy:
# Cross-region replica: ElastiCache global datastore
# Manual: backup to S3, restore in target region
# RT0: 30 min (create new cluster + restore)
# RP0: 1 hour (hourly backups) or minutes (with replica)
```

Q120

What is Redis slow log?

```
# Slow log: commands that exceeded threshold
slowlog-log-slower-than 10000 # microseconds (10ms)
slowlog-max-len 128          # keep last 128 entries

# View slow log
SLOWLOG GET 10 # last 10 slow commands
SLOWLOG LEN    # number in slow log
SLOWLOG RESET  # clear slow log

# Entry format:
# 1) ID
# 2) timestamp
# 3) duration (microseconds)
# 4) command + args

# Common causes:
# KEYS * : O(N) scan of all keys – never use in production
# SMEMBERS on large set: O(N)
# SORT without LIMIT: O(N+M*log(M))
# LRANGE 0 -1 on large list: O(N)

# Use instead:
# SCAN cursor MATCH pattern COUNT 100 # iterative, non-blocking
# SSCAN for sets
# HSCAN for hashes
# ZSCAN for sorted sets
```

Q121–Q150: Remaining Cache Questions

| Q | Topic |

|---|---|

Q121	Redis OBJECT FREQ and LFU eviction
Q122	Cache versioning strategies
Q123	Write coalescing (batch cache writes)
Q124	Redis Sentinel automatic failover
Q125	Cache compression trade-offs
Q126	Redis key expiry notifications in Go
Q127	In-memory cache vs Redis: when to use each
Q128	Cache coherence in microservices
Q129	Redis OBJECT IDLETIME for access patterns
Q130	Distributed caching patterns for APIs
Q131	Cache hit ratio optimization
Q132	Redis Modules: RedisSearch, RedisJSON
Q133	Redis replication buffer tuning
Q134	Cache warming from database dump

Q135	Session caching best practices
Q136	Redis INFO command interpretation
Q137	Cache tier design for e-commerce
Q138	Redis key pattern analysis with redis-cli
Q139	Cache consistency models (strong vs eventual)
Q140	Redis cluster cross-slot operations
Q141	Cache monitoring with Datadog/New Relic
Q142	Redis configuration for low-latency
Q143	Multi-tenant caching (namespace isolation)
Q144	Cache debugging techniques
Q145	Redis Enterprise vs open source Redis
Q146	Cache testing in CI/CD pipelines
Q147	Redis bulk operations (MSET, MGET, pipelines)
Q148	Cache observability (OpenTelemetry)
Q149	Valkey vs Redis (post-license fork)
Q150	Cache production readiness checklist

Q123

What is write-through vs write-behind caching?

Write-Through:

- Write to cache AND DB synchronously
- Cache always consistent with DB
- Higher write latency (both writes must complete)
- Use: read-heavy, consistency required

Write-Behind (Write-Back):

- Write to cache only, async write to DB
- Lower write latency (only cache write blocks)
- Risk: data loss if cache fails before DB write
- Use: write-heavy, eventual consistency ok
- Mitigation: WAL (write-ahead log) in cache

Write-Around:

- Write directly to DB, bypass cache
- Cache populated on next read (cache miss)
- Use: infrequently read data, large objects
- Prevents cache pollution from one-time writes

Cache-Aside (Lazy Loading):

- App manages cache explicitly
- Read: cache → DB on miss → populate cache
- Write: update DB + invalidate cache
- Use: most common pattern, flexible

Read-Through:

- Cache sits in front of DB transparently
- Cache fetches from DB on miss automatically
- App only talks to cache
- Use: simplifies app code, uniform caching layer

Q124

What is Redis keyspace notifications?

```
// Keyspace notifications: get events when keys expire/modified
// Config (redis.conf or SET):
// notify-keyspace-events "Ex"
// E = keyspace events, x = expired events

rdb.Do(ctx, "CONFIG", "SET", "notify-keyspace-events", "Ex")

// Subscribe to expiry notifications
pubsub := rdb.Subscribe(ctx, "__keyevent@0__:expired")
defer pubsub.Close()

ch := pubsub.Channel()
for msg := range ch {
    expiredKey := msg.Payload
    log.Printf("key expired: %s", expiredKey)
    // Trigger: cleanup, re-fetch, notify
}

// Other event types:
// K = keyspace events (channel: __keyspace@<db>__:<key>)
// E = keyevent events (channel: __keyevent@<db>__:<event>)
// g = generic commands (del, expire, rename)
// s = set commands
// l = list commands
// z = sorted set commands
// x = expired events
// d = module key events
// t = stream commands

// notify-keyspace-events "KEA" = all events
// Use carefully: high-volume events can overwhelm Redis
```

Q125

What is Redis memory usage analysis?


```

# Memory info
redis-cli INFO memory

# Key fields:
# used_memory:          total allocated by Redis
# used_memory_rss:      RSS from OS (includes fragmentation)
# mem_fragmentation_ratio: RSS/used (> 1.5 = fragmented, < 1 = swapping)
# maxmemory:            limit (0 = no limit)
# mem_allocator:        jemalloc (default, good for fragmentation)

# Find memory hogs
redis-cli --bigkeys          # top 1 key per type by size
redis-cli MEMORY USAGE mykey # bytes for one key
redis-cli MEMORY DOCTOR      # diagnostic suggestions

# Scan all keys and estimate memory (don't use KEYS * in production!)
redis-cli --scan --pattern "user:*" | xargs -I{} redis-cli MEMORY USAGE {} | awk '{sum+=$1} END{print "Total memory usage: " sum}'

# Key count by pattern (approximate)
redis-cli DBSIZE          # total keys in current db

# Memory breakdown:
# MEMORY MALLOC-STATS    # jemalloc stats
# MEMORY PURGE           # return fragmented memory to OS (Redis 4.0+)

# Per-database key counts (Redis has 16 DBs by default)
redis-cli INFO keyspace

```

Q126

What is Redis OBJECT ENCODING for memory optimization?

```
# Encoding determines memory usage per data type

# String
SET counter 100
OBJECT ENCODING counter      # "int" (very compact!)
SET name "Alice"
OBJECT ENCODING name         # "embstr" (≤ 44 bytes)
SET longtext "...long text..."
OBJECT ENCODING longtext     # "raw"

# Hash
HSET user:1 name Alice age 30
OBJECT ENCODING user:1       # "listpack" (≤ 128 fields and ≤ 64 bytes each)
# Add many fields:
OBJECT ENCODING user:1       # "hashtable" (after threshold exceeded)

# Memory savings (listpack vs hashtable):
# 10-field hash: listpack ~200 bytes, hashtable ~800 bytes

# Tune thresholds for your data:
CONFIG SET hash-max-listpack-entries 256  # allow larger listpack hashes
CONFIG SET hash-max-listpack-value 128    # larger values in listpack

# Sorted Set
ZADD leaderboard 100 "user1"
OBJECT ENCODING leaderboard  # "listpack" or "skiplist"

# Always check OBJECT ENCODING after tuning to verify effect
```

Q127

What is Redis vs Memcached comparison?

Redis:

Data structures: `string`, hash, list, set, zset, stream, geo, bitmap
Persistence: RDB snapshots + AOF log
Replication: async master-replica
Clustering: built-in cluster mode (hash slots)
Pub/Sub: yes
Transactions: MULTI/EXEC (optimistic)
Lua scripting: yes
Max value size: 512MB
Eviction: multiple policies (LRU, LFU, TTL-based)
Use: caching + data structures + sessions + queues + pub/sub

Memcached:

Data structures: `string` only (key-value)
Persistence: NO (in-memory only)
Replication: NO (client-side sharding only)
Clustering: client-side (consistent hashing)
Pub/Sub: NO
Transactions: NO
Lua scripting: NO
Max value size: 1MB
Eviction: LRU only
Use: pure caching (high performance, simple)

Memcached advantages:

Slightly faster for simple `string` ops (less overhead)
Multi-threading (Redis: single-threaded command loop until 6.0)
Simpler: nothing to misconfigure

Recommendation (2025):

Redis for virtually all new projects
Memcached: only for very specific high-throughput pure cache needs

Q128

What is Redis connection pooling in Go?

```

import "github.com/redis/go-redis/v9"

// go-redis: connection pool built-in
rdb := redis.NewClient(&redis.Options{
    Addr:      "localhost:6379",
    Password:  "",
    DB:        0,

    // Pool settings
    PoolSize:    10,                // connections per CPU (default: 10 * runtime.NumCPU())
    MinIdleConns: 5,                // keep N idle connections ready
    MaxIdleConns: 10,               // max idle connections

    // Timeouts
    DialTimeout:  5 * time.Second,
    ReadTimeout:  3 * time.Second,
    WriteTimeout: 3 * time.Second,
    PoolTimeout:  4 * time.Second, // wait for free connection
    ConnMaxIdleTime: 30 * time.Minute, // close idle after
    ConnMaxLifetime: time.Hour,        // max connection age
})

// Health check
ctx := context.Background()
if err := rdb.Ping(ctx).Err(); err != nil {
    log.Fatalf("redis ping failed: %v", err)
}

// Monitor pool stats
stats := rdb.PoolStats()
fmt.Printf("hits=%d misses=%d timeouts=%d totalConns=%d idleConns=%d\n",
    stats.Hits, stats.Misses, stats.Timeouts, stats.TotalConns, stats.IdleConns)

// Alert: Timeouts > 0 sustained → pool exhausted → increase PoolSize

```

Q129

What is Redis WAIT for read-after-write consistency?

```

// Problem: write to master → immediately read from replica → stale data!
// Read-after-write: user writes then immediately reads own data

// Solution 1: always read from master (simplest, correct)
masterRDB := redis.NewClient(&redis.Options{Addr: "master:6379"})

// Solution 2: WAIT for replica sync
func updateAndRead(ctx context.Context, key, value string) (string, error) {
    if err := masterRDB.Set(ctx, key, value, time.Hour).Err(); err != nil {
        return "", err
    }

    // Wait for at least 1 replica to acknowledge the write
    numReplicas, err := masterRDB.Wait(ctx, 1, 100*time.Millisecond).Result()
    if err != nil || numReplicas < 1 {
        // Replica sync timeout - read from master to be safe
        return masterRDB.Get(ctx, key).Result()
    }

    // Safe to read from replica now
    return replicaRDB.Get(ctx, key).Result()
}

// Solution 3: sticky session routing
// User X always reads from same replica after write
// Route by user_id hash: user writes/reads go to same replica

// Solution 4: version tokens
// Return version on write, pass version on read, replica confirms it has version

```

Q130

What is Redis ACL (Access Control Lists)?

```
# Redis 6.0+: fine-grained access control
# Default user: "default" (backward compat, can restrict)

# Create read-only user for application
redis-cli ACL SETUSER api-reader on \ # enabled
    >securepassword \ # password
    ~data:* \ # can access keys matching data:*
    &* \ # can subscribe to all channels
    +GET +MGET +HGET +HGETALL \ # allowed commands
    +INFO +PING \ # monitoring
    -DEBUG # explicit deny

# Create write user for specific namespace
redis-cli ACL SETUSER api-writer on >password ~session:* +SET +GET +DEL +EXPIRE

# Admin user (full access)
redis-cli ACL SETUSER admin on >adminpass ~* &* +@all

# List users
redis-cli ACL LIST
redis-cli ACL WHOAMI # current user
redis-cli ACL CAT # list command categories

# Disable default user (security hardening)
redis-cli ACL SETUSER default off

# In go-redis:
redis.NewClient(&redis.Options{
    Username: "api-reader",
    Password: "securepassword",
})
```

Q131

What is Redis sentinel vs cluster for HA?

Redis Sentinel:

- Purpose: HA for single master + replicas
- Components: N sentinel processes (recommend: 3)
- Failure detection: majority of sentinels agree (quorum=2)
- Failover: sentinel promotes replica → updates clients
- Scale: vertical only (one master handles writes)
- Use: single-region, < 100GB data, moderate throughput

Redis Cluster:

- Purpose: HA + horizontal scaling
- Data split: 16384 hash slots across N masters
- Each master: has 1-N replicas
- Failover: replica promoted automatically
- Scale: add shards (horizontal)
- Client: must be cluster-aware (MOVED/ASK handling)
- Limitation: multi-key ops only within same slot/hash tag
- Use: > 100GB data, high throughput, multi-region

Choosing:

- < 10GB, simple HA → Sentinel
- > 10GB or > 100K ops/sec → Cluster
- Need cross-slot transactions → Sentinel (single master)
- Massive scale → Cluster

Managed options:

- AWS ElastiCache: both Sentinel and Cluster modes
- Redis Cloud: Redis Enterprise (better than open-source cluster)
- Upstash: serverless Redis (pay per request)

Q132

What is Redis rate limiting with Lua?

```
// Lua script: atomic check-and-increment
// Prevents race conditions in distributed rate limiting

const rateLimitScript = `
local key = KEYS[1]
local limit = tonumber(ARGV[1])
local window = tonumber(ARGV[2])
local now = tonumber(ARGV[3])

-- Remove old entries outside window
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)

-- Count current requests
local count = redis.call('ZCARD', key)

if count < limit then
    -- Add this request
    redis.call('ZADD', key, now, now .. math.random())
    redis.call('EXPIRE', key, window / 1000 + 1)
    return 1 -- allowed
end
return 0 -- denied
`

func isAllowed(ctx context.Context, rdb *redis.Client, userID string, limit int, windowMs int64) bool {
    now := time.Now().UnixMilli()
    key := fmt.Sprintf("ratelimit:%s", userID)

    result, err := rdb.Eval(ctx, rateLimitScript,
        []string{key},
        limit, windowMs, now,
    ).Int()

    if err != nil { return true } // fail open
    return result == 1
}
```

Q133

What is Redis caching for database queries?


```
// Pattern: cache DB query results by normalized query hash

type QueryCache struct {
    rdb *redis.Client
    db  *pgxpool.Pool
    ttl time.Duration
}

func (qc *QueryCache) Query(ctx context.Context, sql string, args ...interface{}) ([]map[string]interface{}, error) {
    // Create cache key from query + args
    h := sha256.New()
    fmt.Fprintf(h, "%s:%v", sql, args)
    cacheKey := fmt.Sprintf("query:%x", h.Sum(nil))

    // Check cache
    data, err := qc.rdb.Get(ctx, cacheKey).Bytes()
    if err == nil {
        var rows []map[string]interface{}
        if json.Unmarshal(data, &rows) == nil {
            return rows, nil
        }
    }

    // Execute query
    rows, err := qc.executeQuery(ctx, sql, args...)
    if err != nil { return nil, err }

    // Cache result
    if data, err := json.Marshal(rows); err == nil {
        qc.rdb.Set(ctx, cacheKey, data, qc.ttl)
    }

    return rows, nil
}

// Invalidation: use tags (store set of keys per tag)
func (qc *QueryCache) Invalidate(ctx context.Context, tags ...string) {
    for _, tag := range tags {
        keys, _ := qc.rdb.SMembers(ctx, "tag:"+tag).Result()
        if len(keys) > 0 {
            qc.rdb.Del(ctx, keys...)
        }
        qc.rdb.Del(ctx, "tag:"+tag)
    }
}
}
```

Q134

What is Redis sorted set for leaderboard?

```
// Leaderboard: sorted by score, rank queries efficient

// Add/update score
rdb.ZAdd(ctx, "leaderboard:global", redis.Z{Score: 15420, Member: "user:42"})
rdb.ZIncrBy(ctx, "leaderboard:global", 100, "user:42") // add 100 points

// Get top 10 (highest scores first)
top10, _ := rdb.ZRevRangeWithScores(ctx, "leaderboard:global", 0, 9).Result()
for i, z := range top10 {
    fmt.Printf("#%d: %s → %.0f points\n", i+1, z.Member, z.Score)
}

// Get user's rank (0-indexed)
rank, _ := rdb.ZRevRank(ctx, "leaderboard:global", "user:42").Result()
score, _ := rdb.ZScore(ctx, "leaderboard:global", "user:42").Result()
fmt.Printf("user:42 rank=%d score=%.0f\n", rank+1, score)

// Get neighborhood (user's rank ± 2)
rdb.ZRevRangeWithScores(ctx, "leaderboard:global", rank-2, rank+2)

// Weekly leaderboard (expire after 1 week)
weekKey := fmt.Sprintf("leaderboard:week:%s", time.Now().Format("2006-W01"))
rdb.ZAdd(ctx, weekKey, redis.Z{Score: points, Member: userID})
rdb.Expire(ctx, weekKey, 8*24*time.Hour)

// Count users above threshold
rdb.ZCount(ctx, "leaderboard:global", "1000", "+inf")
```

Q135

What is Redis HyperLogLog for cardinality?

```
// HyperLogLog: probabilistic cardinality estimation
// Estimate unique count with ~0.81% error, uses only 12KB per HLL

// Count unique visitors to a page
rdb.PFAdd(ctx, "visitors:2024-01-15", "user:123", "user:456", "user:789")
rdb.PFAdd(ctx, "visitors:2024-01-15", "user:123") // duplicate, not counted

// Get unique count
count, _ := rdb.PFCount(ctx, "visitors:2024-01-15").Result()
fmt.Printf("unique visitors: %d", count) // approximately 3

// Merge multiple HLLs (e.g., weekly from daily)
rdb.PFMerge(ctx, "visitors:2024-W03",
    "visitors:2024-01-15",
    "visitors:2024-01-16",
    "visitors:2024-01-17",
)

weeklyUnique, _ := rdb.PFCount(ctx, "visitors:2024-W03").Result()

// Comparison vs SADD (exact):
// HLL: 12KB for any cardinality, ~1% error
// SET: N bytes per element, exact count
// Use HLL for: daily active users, search queries, A/B test reach
// Use SET for: must-be-exact, small cardinality (< 100K elements)
```

Q136

What is Redis pipeline vs transaction?

```
// Pipeline: batch commands, reduce RTT (NOT atomic)
pipe := rdb.Pipeline()
pipe.Set(ctx, "key1", "val1", time.Hour)
pipe.Set(ctx, "key2", "val2", time.Hour)
pipe.Get(ctx, "key1")
cmds, err := pipe.Exec(ctx)
// All 3 commands sent in 1 RTT, but if key1 SET fails, key2 SET still runs

// TxPipeline: pipeline + MULTI/EXEC (atomic)
// If any command fails during EXEC, all commands rolled back
txPipe := rdb.TxPipeline()
txPipe.Set(ctx, "key1", "val1", time.Hour)
txPipe.Set(ctx, "key2", "val2", time.Hour)
_, err = txPipe.Exec(ctx)
// Atomic: either both set or neither

// WATCH + MULTI/EXEC (optimistic locking):
err = rdb.Watch(ctx, func(tx *redis.Tx) error {
    val, err := tx.Get(ctx, "balance").Int()
    if err != nil { return err }

    if val < 100 { return errors.New("insufficient funds") }

    _, err = tx.TxPipelined(ctx, func(pipe redis.Pipeliner) error {
        pipe.Set(ctx, "balance", val-100, 0)
        return nil
    })
    return err
}, "balance")
// If "balance" changed between WATCH and EXEC → retried (optimistic)
```

Q137

What is Redis Bloom filter?

```

# Bloom filter: probabilistic set membership test
# False positives possible, false negatives impossible
# Extremely memory efficient

# Requires RedisBloom module (or Redis Stack)
BF.RESERVE emails 0.01 1000000 # 1% false positive rate, 1M elements
BF.ADD emails "user@example.com"
BF.EXISTS emails "user@example.com" # 1 = likely member, 0 = definitely not

# Use case: prevent duplicate email registration
func canRegister(ctx context.Context, email string) bool {
    exists, _ := rdb.Do(ctx, "BF.EXISTS", "registered-emails", email).Int()
    return exists == 0 // definitely not registered (bloom filter says so)
}

func registerUser(ctx context.Context, email string) error {
    // ... create user in DB ...
    rdb.Do(ctx, "BF.ADD", "registered-emails", email)
    return nil
}

# Cuckoo filter (better: supports deletion)
CF.RESERVE urls 1000000
CF.ADD urls "https://example.com"
CF.DEL urls "https://example.com" # deletion supported!
CF.EXISTS urls "https://example.com"

```

Q138

What is Redis Streams consumer groups in Go?

```
// Create consumer group
rdb.XGroupCreate(ctx, "events", "processors", "0") // "0" = start from beginning
// "$" = only new messages

// Producer: add to stream
rdb.XAdd(ctx, &redis.XAddArgs{
    Stream: "events",
    Values: map[string]interface{}{"type": "order.placed", "id": "42"},
})

// Consumer: read from group
result, _ := rdb.XReadGroup(ctx, &redis.XReadGroupArgs{
    Group:    "processors",
    Consumer: "worker-1",
    Streams:  []string{"events", ">"}, // ">" = undelivered to this group
    Count:    10,
    Block:    2 * time.Second,
}).Result()

for _, stream := range result {
    for _, msg := range stream.Messages {
        if err := process(msg); err == nil {
            rdb.XAck(ctx, "events", "processors", msg.ID) // acknowledge
        }
        // No ACK: message stays in PEL (pending entry list)
    }
}

// Claim stuck messages (consumer crashed without ACK)
pending, _ := rdb.XPendingExt(ctx, &redis.XPendingExtArgs{
    Stream: "events", Group: "processors",
    Idle: 5 * time.Minute, Start: "-", End: "+", Count: 10,
}).Result()
// Claim: rdb.XClaim → reassign to current consumer
```

Q139

What is multi-level caching architecture?

```
L1: In-process cache (ristretto/bigcache)
  Latency: ~100ns
  Size: 100MB-1GB (bounded by process memory)
  Consistency: per-instance (different servers may have different values)
  TTL: 30s-5min (short, accept stale)

L2: Redis (shared cache)
  Latency: <1ms (same datacenter)
  Size: GBs (Redis Cluster for TBs)
  Consistency: shared across all instances
  TTL: minutes-hours

L3: Database (source of truth)
  Latency: 5-50ms
  Consistency: always correct
  Capacity: unlimited (with sharding)

Request flow:
  L1 hit → return (fastest)
  L1 miss → L2 hit → populate L1 → return
  L2 miss → DB query → populate L2 → populate L1 → return

Invalidation complexity:
  L1: invalidate per-process (hard without messaging)
  Strategy: short L1 TTL (accept brief stale) + invalidate on write
  Redis Pub/Sub: broadcast invalidation to all processes

When to use multi-level:
  Very high read throughput (> 100K req/s)
  Same data accessed repeatedly in request burst
  L2 latency still too high for hot path
```

Q140

What is Redis for session storage?

```
// Redis: ideal for distributed session storage
// Shared across all app instances, TTL auto-cleanup

type SessionStore struct {
    rdb *redis.Client
    ttl time.Duration
}

func (s *SessionStore) Set(ctx context.Context, sessionID string, data interface{}) error {
    b, err := json.Marshal(data)
    if err != nil { return err }
    return s.rdb.Set(ctx, "session:"+sessionID, b, s.ttl).Err()
}

func (s *SessionStore) Get(ctx context.Context, sessionID string, out interface{}) error {
    b, err := s.rdb.Get(ctx, "session:"+sessionID).Bytes()
    if errors.Is(err, redis.Nil) { return ErrSessionNotFound }
    if err != nil { return err }
    return json.Unmarshal(b, out)
}

// Extend session TTL on activity
func (s *SessionStore) Touch(ctx context.Context, sessionID string) error {
    return s.rdb.Expire(ctx, "session:"+sessionID, s.ttl).Err()
}

func (s *SessionStore) Delete(ctx context.Context, sessionID string) error {
    return s.rdb.Del(ctx, "session:"+sessionID).Err()
}

// Session ID: generate with crypto/rand
sessionID := make([]byte, 32)
rand.Read(sessionID)
id := base64.URLEncoding.EncodeToString(sessionID)
```

Q141–Q150: Final Cache Questions

Q141

What is Redis info command key metrics?

```
redis-cli INFO all # all sections

# Server: version, uptime, OS, tcp port
# Clients: connected_clients, blocked_clients
# Memory: used_memory, maxmemory, mem_fragmentation_ratio
# Persistence: rdb_changes_since_last_save, aof_enabled
# Stats: keyspace_hits, keyspace_misses, evicted_keys
# Replication: role, connected_slaves, master_replid, master_repl_offset
# CPU: used_cpu_sys, used_cpu_user
# Keyspace: db0:keys=100000,expires=50000,avg_ttl=3600000

hit_rate = keyspace_hits / (keyspace_hits + keyspace_misses)
```


Q142

What is Valkey vs Redis?

Redis relicensed (March 2024): SSPL + RSALv2 (not OSI-approved open source)
Valkey: Linux Foundation fork of Redis 7.2 (truly open source, BSD-3)

Valkey 7.2: feature-compatible with Redis 7.2

Valkey 8.0: new features

- Multi-threaded I/O (significant performance improvement)
- Performance improvements
- Active community (AWS, Google, Snap, Ericsson)

go-redis: supports both Redis and Valkey (same protocol)

AWS ElastiCache: supports Valkey as drop-in Redis replacement

DigitalOcean, Aiven: Valkey support added

Migration:

Valkey is wire-compatible with Redis

Same commands, same protocol (RESP2/RESP3)

go-redis client works with both

Drop-in replacement for most use cases

Which to use (2025):

New projects: Valkey (truly OSS, growing community)

Existing Redis: evaluate based on licensing requirements

Redis Cloud/Enterprise: Redis (proprietary features like active-active)

Q143

What is cache stampede prevention with probabilistic early expiration?

```
// XFetch algorithm: probabilistically refresh before expiry
// Avoids thundering herd on cache expiry

func fetchWithPER(ctx context.Context, key string, delta float64, fetchFn func() ([]byte, error)) ([]byte, error) {
    // Try cache first
    pipe := rdb.Pipeline()
    getCmd := pipe.Get(ctx, key)
    ttlCmd := pipe.TTL(ctx, key)
    pipe.Exec(ctx)

    data, err := getCmd.Bytes()
    ttl, _ := ttlCmd.Result()

    if err == nil && ttl > 0 {
        // Probabilistic early recomputation:
        // Refresh if: -delta * ln(random) > remaining_ttl
        remaining := ttl.Seconds()
        if -delta * math.Log(rand.Float64()) > remaining {
            // Eagerly refresh (beat the expiry)
            goto refresh
        }
        return data, nil
    }

refresh:
    fresh, err := fetchFn()
    if err != nil {
        if data != nil { return data, nil } // serve stale on error
        return nil, err
    }
    rdb.Set(ctx, key, fresh, time.Hour)
    return fresh, nil
}

// delta: typical recomputation time in seconds
// Higher delta = earlier probabilistic refresh
```

Q144

What is cache tagging for bulk invalidation?

```
// Problem: how to invalidate all caches related to "user:42"?
// Solution: tag keys and invalidate by tag

func setCacheWithTags(ctx context.Context, key string, data []byte, ttl time.Duration, tags ...string) error {
    pipe := rdb.Pipeline()
    pipe.Set(ctx, key, data, ttl)
    for _, tag := range tags {
        pipe.SAdd(ctx, "tag:"+tag, key)
        pipe.Expire(ctx, "tag:"+tag, ttl+time.Minute)
    }
    _, err := pipe.Exec(ctx)
    return err
}

func invalidateByTag(ctx context.Context, tag string) error {
    keys, err := rdb.SMembers(ctx, "tag:"+tag).Result()
    if err != nil { return err }
    if len(keys) > 0 {
        keys = append(keys, "tag:"+tag)
        return rdb.Del(ctx, keys...).Err()
    }
    return rdb.Del(ctx, "tag:"+tag).Err()
}

// Usage
setCacheWithTags(ctx, "user:42:profile", data, time.Hour, "user:42", "profiles")
setCacheWithTags(ctx, "user:42:orders", orderData, time.Hour, "user:42", "orders")

// On user update: invalidate everything for user:42
invalidateByTag(ctx, "user:42") // deletes both profile and orders cache
```

Q145

What is Redis for distributed locks in Go?

```

// Simple distributed lock (single Redis instance)
func acquireLock(ctx context.Context, rdb *redis.Client, key, token string, ttl time.Duration) (bool, error) {
    // SET key token NX PX ttl_ms (atomic: only set if not exists)
    ok, err := rdb.SetNX(ctx, "lock:"+key, token, ttl).Result()
    return ok, err
}

func releaseLock(ctx context.Context, rdb *redis.Client, key, token string) error {
    // Lua: only delete if we own the lock (check token)
    script := `
        if redis.call("get", KEYS[1]) == ARGV[1] then
            return redis.call("del", KEYS[1])
        else
            return 0
        end
    `
    return rdb.Eval(ctx, script, []string{"lock:" + key}, token).Err()
}

// Usage
token := uuid.New().String()
acquired, err := acquireLock(ctx, rdb, "resource-123", token, 30*time.Second)
if !acquired { return ErrLockNotAcquired }
defer releaseLock(ctx, rdb, "resource-123", token)

// Process critical section
doWork()

// For multi-node: use Redlock (redsync library)

```

Q146

What is cache warm-up strategies?

```
// Problem: cold start → all requests miss cache → overload DB

// Strategy 1: Pre-warm from DB on startup
func warmCache(ctx context.Context, rdb *redis.Client, db *pgxpool.Pool) error {
    rows, err := db.Query(ctx, "SELECT id, data FROM products WHERE active=true ORDER BY view_count DESC LIMIT 1000")
    if err != nil { return err }
    defer rows.Close()

    pipe := rdb.Pipeline()
    for rows.Next() {
        var id int64
        var data []byte
        rows.Scan(&id, &data)
        pipe.Set(ctx, fmt.Sprintf("product:%d", id), data, 24*time.Hour)
    }
    _, err = pipe.Exec(ctx)
    return err
}

// Strategy 2: Shadow traffic (replay prod traffic to new instance)
// Strategy 3: Cache seeding from S3 snapshot
// Strategy 4: Gradual rollout (small % traffic first)
// Strategy 5: Request coalescing (singleflight during warm-up)

var sfGroup singleflight.Group
func getProduct(ctx context.Context, id int64) (*Product, error) {
    result, err, _ := sfGroup.Do(fmt.Sprintf("product:%d", id), func() (interface{}, error) {
        return fetchFromDB(ctx, id)
    })
    return result.(*Product), err
}
```

Q147

What is Redis for feature flags?

```
// Feature flags stored in Redis: instant rollout without deploy

type FeatureFlags struct {
    rdb *redis.Client
}

// Set feature flag
func (ff *FeatureFlags) Enable(ctx context.Context, flag string) error {
    return ff.rdb.Set(ctx, "feature:"+flag, "true", 0).Err()
}
func (ff *FeatureFlags) Disable(ctx context.Context, flag string) error {
    return ff.rdb.Del(ctx, "feature:"+flag).Err()
}
func (ff *FeatureFlags) IsEnabled(ctx context.Context, flag string) bool {
    val, _ := ff.rdb.Get(ctx, "feature:"+flag).Result()
    return val == "true"
}

// Percentage rollout
func (ff *FeatureFlags) IsEnabledForUser(ctx context.Context, flag string, userID int64) bool {
    // Get rollout percentage
    pct, err := ff.rdb.Get(ctx, "feature:"+flag+":pct").Int()
    if err != nil { return false }
    // Deterministic: same user always gets same result
    return (userID % 100) < int64(pct)
}

// Listen for changes (keyspace notifications)
func (ff *FeatureFlags) WatchChanges(ctx context.Context, onChange func(flag string)) {
    pubsub := ff.rdb.Subscribe(ctx, "__keyevent@0__:set", "__keyevent@0__:del")
    for msg := range pubsub.Channel() {
        if strings.HasPrefix(msg.Payload, "feature:") {
            onChange(strings.TrimPrefix(msg.Payload, "feature:"))
        }
    }
}
```

Q148

What is Redis for search autocomplete?

```
// Autocomplete using sorted set with score=0 trick (lexicographic range)

// Index: ZADD index 0 "apple" 0 "application" 0 "apply"
func indexWord(ctx context.Context, rdb *redis.Client, word string) error {
    return rdb.ZAdd(ctx, "autocomplete", redis.Z{Score: 0, Member: word}).Err()
}

// Search prefix
func autocomplete(ctx context.Context, rdb *redis.Client, prefix string, limit int) ([]string, error) {
    min := "[" + prefix // inclusive lower bound
    max := "[" + prefix + "ÿ" // prefix + max char (all strings starting with prefix)

    results, err := rdb.ZRangeByLex(ctx, "autocomplete", &redis.ZRangeBy{
        Min: min,
        Max: max,
        Count: int64(limit),
    }).Result()
    return results, err
}

// With frequency scoring (better UX)
func recordSearch(ctx context.Context, rdb *redis.Client, query string) error {
    return rdb.ZIncrBy(ctx, "search:frequency", 1, query).Err()
}

func autocompleteByFrequency(ctx context.Context, rdb *redis.Client, prefix string) ([]string, error) {
    // Use trie or scan-based approach for prefix + score sort
    // Or: maintain separate sorted set per first character
    return rdb.ZRevRangeByScore(ctx, "search:frequency:"+prefix[0:1],
        &redis.ZRangeBy{Min: "-inf", Max: "+inf", Count: 10}).Result()
}
```

Q149

What is Redis for job queues?

```
// Redis as job queue (simple, without Kafka/RMQ overhead)

// Enqueue job
func enqueue(ctx context.Context, rdb *redis.Client, queue, jobJSON string) error {
    return rdb.LPush(ctx, "queue:"+queue, jobJSON).Err()
}

// Dequeue (blocking pop, waits for jobs)
func dequeue(ctx context.Context, rdb *redis.Client, queue string) (string, error) {
    result, err := rdb.BRPop(ctx, 10*time.Second, "queue:"+queue).Result()
    if err != nil { return "", err }
    return result[1], nil // result[0] is queue name, result[1] is value
}

// Reliable queue (BRPOPLUSH: atomic move to processing list)
func dequeueReliable(ctx context.Context, rdb *redis.Client, queue string) (string, error) {
    return rdb.BRPopLPush(ctx, "queue:"+queue, "queue:"+queue+":processing", 10*time.Second).Result()
}

// Acknowledge (remove from processing list)
func ack(ctx context.Context, rdb *redis.Client, queue, job string) error {
    return rdb.LRem(ctx, "queue:"+queue+":processing", 1, job).Err()
}

// Recover stale jobs (crashed workers)
func recoverStale(ctx context.Context, rdb *redis.Client, queue string) error {
    jobs, _ := rdb.LRange(ctx, "queue:"+queue+":processing", 0, -1).Result()
    for _, job := range jobs {
        rdb.RPush(ctx, "queue:"+queue, job)
    }
    return rdb.Del(ctx, "queue:"+queue+":processing").Err()
}
```

Q150

What is cache production readiness checklist?

Configuration:

- maxmemory set (never let Redis use unlimited memory)
- maxmemory-policy = allkeys-lru (or allkeys-lfu)
- Persistence configured (RDB for snapshots, AOF for durability)
- bind configured (not exposed to public internet)
- requirepass set (authentication)
- ACL configured (least privilege)
- notify-keyspace-events disabled (unless needed, adds overhead)

HA:

- Sentinel (3 nodes) or Cluster mode for HA
- Replica in separate AZ
- Automatic failover tested

Monitoring:

- Prometheus exporter (redis-exporter)
- Alert: used_memory > 80% maxmemory
- Alert: evicted_keys > 0 (cache under pressure)
- Alert: connected_clients approaching max
- Alert: replication_lag > 10s
- Hit rate dashboard: $\text{keyspace_hits} / (\text{hits} + \text{misses})$

Application:

- Connection pool properly sized
- Timeouts configured (read/write/pool)
- Retry logic with exponential backoff
- Circuit breaker (fail gracefully without cache)
- Cache key namespacing (environment prefix)
- TTL on all keys (no immortal keys)
- Singleflight for thundering herd prevention

Security:

- TLS enabled (Redis 6.0+)
- Redis not exposed to internet
- No sensitive data in keys (PII in values only, encrypted if needed)